

Trabajo de Fin de Grado

Grado en Ingeniería Informática

Análisis de Capacidades de la IA en la Generación Semiautomática de Aplicaciones Complejas

*Estudio del código generado por modelos de Inteligencia
Artificial*

Fabián González Lence

La Laguna, 29 de mayo de 2026

D. **Francisco de Sande González**, profesor Titular de Universidad adscrito al Departamento de Ingeniería Informática y de Sistemas de la Universidad de La Laguna, en calidad de tutor

C E R T I F I C A

Que el presente trabajo de Fin de Grado titulado:

“Análisis de Capacidades de la IA en la Generación Semiautomática de Aplicaciones Complejas”

ha sido realizado bajo su dirección por D. **Fabián González Lence**.

Y para que así conste, en cumplimiento de la legislación vigente y a los efectos oportunos firman la presente memoria del Trabajo en La Laguna a 29 de mayo de 2026.

Agradecimientos

Quiero agradecer a Francisco de Sande, por su guía, disponibilidad y orientación a lo largo de todo el desarrollo de este trabajo. Sus consejos y el rigor con el que ha dirigido el proyecto han sido fundamentales para llevar esta investigación a buen puerto.

A la Universidad de La Laguna y a la Escuela Superior de Ingeniería y Tecnología, por la formación recibida durante estos años y por los recursos puestos a disposición del alumnado.

A mi familia y amigos, por el apoyo incondicional durante toda la carrera y, especialmente, en estos últimos meses de trabajo intenso.

Por último, a todas las personas que, de una forma u otra, han contribuido a que este proyecto saliera adelante.

Este trabajo ha sido financiado por el Ministerio de Ciencia e Innovación a través del proyecto PID2023-151073NB-I00 Aceleración y Optimización en Aprendizaje Automático: Análisis de Rendimiento y Energía

Licencia



© Esta obra está bajo una licencia de Creative Commons Reconocimiento-NoComercial-CompartirIgual 4.0 Internacional.

Resumen

El rápido avance de los grandes modelos de lenguaje (LLMs) ha abierto nuevas posibilidades en el ámbito del desarrollo de software asistido por inteligencia artificial. Este Trabajo de Fin de Grado analiza las capacidades de distintos modelos de IA (Inteligencia Artificial) generativa para dar soporte a las diferentes fases del ciclo de vida del desarrollo de software de forma colaborativa y semiautomática.

Con el objetivo de evaluar de manera sistemática el rendimiento de estos modelos, se ha diseñado y seguido una metodología experimental basada en las fases clásicas del desarrollo de software: especificación de requisitos, diseño, implementación, revisión, pruebas y despliegue. Dicha metodología se ha aplicado al desarrollo de cinco aplicaciones de complejidad creciente: un juego del ahorcado, un reproductor web de música, una versión simplificada del videojuego Balatro, un gestor de proyectos cartográficos y un gestor de torneos de tenis.

Para cada aplicación, se han generado múltiples versiones de los artefactos de desarrollo —requisitos en lenguaje natural, semiformales y en lenguaje controlado, diagramas UML y código fuente— utilizando distintos modelos de IA como agentes especializados. El proceso ha sido documentado rigurosa y exhaustivamente, registrando los prompts empleados, los resultados obtenidos y las decisiones tomadas en cada etapa.

Los resultados obtenidos permiten valorar el grado de madurez actual de los LLMs en tareas de ingeniería del software, identificar sus fortalezas y limitaciones en contextos reales de desarrollo, y extraer conclusiones sobre el potencial del enfoque colaborativo multiagente como paradigma emergente en la generación semiautomática de aplicaciones.

Palabras clave: Inteligencia Artificial, IA Generativa, Modelos de Lenguaje de Gran Tamaño, Generación de Código, Ingeniería del Software, Desarrollo Asistido por IA, Calidad del Software, Colaboración Multiagente, Prompt Engineering.

Abstract

The rapid advancement of large language models (LLMs) has opened new possibilities in the field of AI-assisted software development. This Bachelor's Thesis analyses the capabilities of several generative AI models in supporting the different phases of the software development life cycle in a collaborative and semi-automatic manner.

In order to systematically evaluate the performance of these models, an experimental methodology based on the classical phases of software engineering has been designed and followed: requirements specification, design, implementation, review, testing and deployment. This methodology has been applied to the development of five applications of increasing complexity: a Hangman game, a web music player, a simplified version of the Balatro card game, a cartographic project manager and a tennis tournament manager.

For each application, multiple versions of the development artefacts have been generated—requirements in natural language, semi-formal and controlled language, UML diagrams and source code—using different AI models as specialised agents. The process has been rigorously and thoroughly documented, recording the prompts used, the results obtained and the decisions made at each stage.

The results obtained allow us to assess the current maturity of LLMs in software engineering tasks, identify their strengths and limitations in real development contexts, and draw conclusions about the potential of the collaborative multi-agent approach as an emerging paradigm in the semi-automatic generation of applications.

Keywords: Artificial Intelligence, Generative AI, Large Language Models, Code Generation, Software Engineering, AI-Assisted Development, Software Quality, Multi-Agent Collaboration, Prompt Engineering.

Índice general

1	Introducción	1
1.1	Objetivos	1
1.2	Motivación y contexto	2
1.3	Metodología general	3
1.4	Calidad y trazabilidad	4
2	Estado del Arte	5
2.1	Generación de código con LLMs	5
2.2	Evaluación del código generado por IA	5
2.3	Colaboración multiagente en el desarrollo	6
2.4	Posicionamiento del presente trabajo	7
3	Modelos de IA	8
3.1	Panorama actual de los LLMs en el desarrollo de software	8
3.2	Modelos y herramientas utilizados	8
3.2.1	Claude	9
3.2.2	Mistral AI	9
3.2.3	Qwen AI	9
3.2.4	ChatGPT	9
3.2.5	Google Gemini y Perplexity — Uso puntual	10
3.2.6	GitHub Copilot	10
3.3	El cambio de enfoque metodológico	10
3.4	Comparativa de modelos	11
3.5	Uso de IA en la redacción de este documento	11
4	Metodología	12
4.1	Diseño experimental	12
4.2	Roles de IA y flujo de trabajo	13
4.3	Plantillas y estrategia de <i>prompting</i>	14
5	Arquitectura y Diseño	16
5.1	Especificación de requisitos	16
5.2	Diagramas de clases y de casos de uso	17
5.3	Decisiones arquitectónicas	18
5.4	Pila tecnológica y trazabilidad de decisiones	18
6	Desarrollo	19
6.1	Fase de Arquitectura	19
6.2	Fase de Codificación	19
6.2.1	P1–P3: codificación modular con Mistral	19
6.2.2	P4–P5: codificación basada en agentes	20
6.3	Fase de Revisión	22
6.3.1	P1–P3: revisión por módulos	22
6.3.2	P4–P5: metodología de tres etapas	22

6.4	Fase de Pruebas	23
6.4.1	P1–P3: Pruebas unitarias con Jest	23
6.4.2	P4–P5: Playwright <i>end-to-end</i>	23
6.5	Retroalimentación de los clientes	24
7	Resultados	25
7.1	Despliegue de los proyectos	25
7.2	Métricas de calidad del código	25
7.3	Análisis de los proyectos resultantes	27
8	Conclusiones y Líneas de Trabajo Futuras	28
8.1	Conclusiones	28
8.2	Líneas de Trabajo Futuro	29
9	Conclusions and Future Lines of Work	30
9.1	Conclusions	30
9.2	Future Lines of Work	31
10	Presupuesto	32
10.1	Horas de trabajo	32
10.2	Suscripciones a servicios de IA	32
10.3	Coste total	33
	Lista de Acrónimos	34
	Bibliografía	35

Capítulo 1: Introducción

1.1 Objetivos

El objetivo principal de este Trabajo de Fin de Grado (TFG) es evaluar de forma práctica las capacidades actuales de la Inteligencia Artificial (IA) generativa para desarrollar aplicaciones no triviales, analizando las prestaciones y rendimiento de varios *Large Language Models* (LLMs) en las diferentes etapas del ciclo de vida del desarrollo de software, *Software Development Life Cycle* (SDLC). Para alcanzar dicho objetivo general, se plantean los siguientes objetivos específicos:

1. **Diseñar una metodología experimental basada en roles de IA.** Definir un flujo de trabajo en el que distintos modelos de IA asuman roles especializados —arquitecto, desarrolladora, revisora de código y generador de pruebas— y cooperen de forma coordinada a lo largo del Ciclo de Vida del Desarrollo de Software.
2. **Desarrollar secuencialmente en el tiempo, cinco aplicaciones de complejidad creciente.** Construir, mediante la colaboración entre modelos de IA las siguientes aplicaciones:
 - (a) **El Juego del Ahorcado (HANGMAN):** Versión web del conocido juego del ahorcado implementado como una *Single Page Application* (SPA), implementando el patrón *Model-View-Controller* (MVC) [1].
 - (b) **Reproductor Web de Música (PLAYER):** Reproductor de música web interactivo simple desarrollado con React¹, TypeScript² y Vite³.
 - (c) **Mini Balatro (BALATRO):** Juego de cartas inspirado en Balatro⁴ con sistema de progresión por niveles, interacciones con una tienda y utilización de manos de póker para el cálculo de puntos.
 - (d) **Aplicación de Gestión de Proyectos Cartográficos (CARTO):** Sistema de gestión de proyectos cartográficos implementando Clean Architecture [2], Vue.js⁵ y comunicación en tiempo real.
 - (e) **Aplicación de Gestion de Torneos de Tenis (TENNIS):** Aplicación de gestión integral de torneos de tenis con múltiples roles de usuario construida en Angular⁶.
3. **Documentar el proceso de interacción con las IAs.** Documentar de forma sistemática los *prompts* empleados, los resultados obtenidos y las decisiones tomadas en cada fase del desarrollo, de modo que el proceso sea reproducible y analizable.
4. **Aplicar un proceso sistemático de revisión y pruebas asistido por IA.** Someter el código generado a revisiones de calidad y a la generación automatizada de conjuntos de pruebas, evaluando la capacidad de los modelos para detectar defectos en código producido por otros modelos.
5. **Evaluar la calidad del software resultante.** Analizar los proyectos desarrollados

¹React – es.react.dev

²TypeScript – typescriptlang.org

³Vite – es.vite.dev

⁴Balatro – es.wikipedia.org/wiki/Balatro

⁵Vue – vuejs.org

⁶Angular – angular.dev

aplicando métricas de calidad de código —funcionalidad, mantenibilidad, coherencia, cobertura de pruebas y adherencia a buenas prácticas—.

6. **Extraer conclusiones sobre la viabilidad del enfoque colaborativo multimodelo.** Sintetizar los hallazgos obtenidos para determinar en qué medida la cooperación entre múltiples IAs especializadas puede sustituir o complementar el trabajo humano en el proceso de desarrollo de software, y proponer líneas futuras de investigación.

1.2 Motivación y contexto

La Inteligencia Artificial (IA), y en particular la IA generativa, está redefiniendo el ciclo de vida del desarrollo de software (SDLC). Lo que hace apenas unos años se consideraba un complemento experimental se ha convertido en un activo central que potencia entornos de desarrollo inteligentes, capaces de automatizar tareas clave, mejorar la eficiencia, acelerar los ciclos de entrega y aumentar la calidad del producto final.

El impacto de las herramientas asistidas por IA en la industria del software es especialmente significativo porque se trata de uno de los dominios en los que estas tecnologías obtienen posiblemente sus mejores resultados. Las estadísticas respaldan esta afirmación: Según encuestas e informes recientes del sector^{1 2}, la adopción de herramientas de IA en el desarrollo de software actualmente es algo generalizado. Por ejemplo, una encuesta de Stack Overflow¹ realizada en 2025 indica que aproximadamente el 84% de los desarrolladores ya utilizan o planean utilizar herramientas de IA en sus flujos de trabajo, mientras que el informe *State of Developer Ecosystem* de JetBrains² sitúa esta cifra en torno al 85%.

El alcance de la IA en el SDLC trasciende la mera generación de código. En las fases iniciales de planificación e ingeniería de requisitos, los modelos de aprendizaje automático facilitan estimaciones de esfuerzo más precisas, mientras que el procesamiento de lenguaje natural permite generar historias de usuario, detectar inconsistencias en requisitos y convertir especificaciones a formatos accionables.

En la fase de diseño, los LLMs pueden proponer candidatos a arquitecturas de software y patrones de diseño a partir de requisitos, así como generar maquetas de interfaz y modelos de datos. En la fase de desarrollo, herramientas como GitHub Copilot⁷, OpenAI Codex⁸ o DeepMind AlphaCode⁹ convierten descripciones en lenguaje natural en código ejecutable e integran asistencia en tiempo real dentro de los entornos de desarrollo. En la fase de pruebas, los algoritmos de IA generan casos de prueba, detectan defectos y optimizan la cobertura sin intervención humana. Incluso en las operaciones de despliegue, la IA automatiza la generación de flujos de trabajo automatizados para las aplicaciones mediante el uso de GitHub Actions¹⁰, la monitorización del rendimiento y la detección de anomalías.

Sin embargo, las investigaciones realizadas se han centrado mayoritariamente en evaluar el rendimiento individual de los modelos en tareas aisladas, prestando particular atención a la generación de código [3, 4]. El potencial de colaboración entre múltiples Inteligencias Artificiales (IAs) especializadas a lo largo del ciclo completo de desarrollo sigue siendo un terreno poco explorado [5]. ¿Qué ocurre cuando se asignan roles diferenciados a distintos modelos —uno como arquitecto, otro como desarrollador, otro como revisor de código y otro como generador

¹Stack Overflow 2025 Developer Survey – survey.stackoverflow.co/2025

²JetBrains State of Developer Ecosystem 2025 – jetbrains.com/lp/devecosystem-2025

⁷GitHub Copilot – github.com/features/copilot

⁸OpenAI Codex – openai.com/codex

⁹DeepMind AlphaCode – deepmind.google

¹⁰GitHub Actions – github.com/features/actions

de pruebas— y se les hace cooperar de forma coordinada para construir aplicaciones no triviales? Esta es una de las preguntas que motivan este trabajo.

Precisamente en aportar información en el contexto antes señalado se enmarca este Trabajo de Fin de Grado, que propone analizar cómo diferentes agentes de IA pueden cooperar de forma complementaria para desarrollar software de manera integrada, evaluando la coherencia, calidad y eficacia del producto resultante. Para ello, se diseña un experimento en el que cinco proyectos web de complejidad creciente —desde un juego sencillo hasta un sistema de gestión de torneos de tenis— son contruidos casi en su totalidad por herramientas de IA, con intervención humana limitada a la supervisión, la validación y la corrección de errores puntuales, sin llegar a intervenir en el desarrollo de código.

1.3 Metodología general

La metodología adoptada en este trabajo se articula en torno a dos ejes complementarios: un diseño experimental basado en proyectos de complejidad creciente y un flujo de trabajo multimodelo en el que distintos modelos de IA asumen roles especializados dentro del SDLC.

Se han definido cinco aplicaciones que, ordenadas de menor a mayor complejidad arquitectónica, funcional y tecnológica se referencian como **HANGMAN**, **PLAYER**, **BALATRO**, **CARTO** y **TENNIS**. El resultado final de cada una de ellas está disponible para su evaluación a través de una página pública [6]. El desarrollo de los cinco proyectos responde no solo a una secuencia creciente en complejidad sino también temporal: **HANGMAN** el más simple, fue el primer proyecto desarrollado y **TENNIS** siendo el más complejo, el último. La secuenciación de las cinco aplicaciones permite observar cómo escalan las capacidades y limitaciones de los modelos a medida que aumentan las exigencias, desde una SPA con patrón MVC hasta un sistema multirrol con comunicación en tiempo real. Para cada proyecto, el proceso de desarrollo se estructura en cuatro fases inspiradas en el SDLC tradicional, cada una asignada a un modelo o herramienta de IA seleccionado:

1. **Arquitectura y diseño:** Se redacta la especificación de requisitos, y a partir de esta, el modelo genera la estructura del proyecto, los diagramas de clases, los casos de uso y la documentación detallada de cada módulo.
2. **Codificación:** Utilizando como entrada los artefactos de diseño producidos en la fase anterior, el modelo genera el código fuente completo de cada módulo, siguiendo las restricciones y los estándares definidos en el *prompt*.
3. **Revisión de código:** El código generado se somete a un proceso de revisión automatizada mediante *prompts* estructurados que evalúan adherencia al diseño, calidad del código, cumplimiento de requisitos, mantenibilidad y buenas prácticas, produciendo un informe con recomendaciones de mejora.
4. **Testing:** A partir del código revisado y aprobado, se genera un conjunto completo de pruebas unitarias con Jest¹¹ o Playwright¹², verificando casos normales, casos límite, casos excepcionales y escenarios de integración.

Este flujo de trabajo reproduce, de forma deliberada, la dinámica de un equipo de desarrollo en el que cada miembro posee una especialización diferente, con la particularidad de que todos los miembros son modelos de IA. La intervención humana se limita a la asistencia en la redacción de las especificaciones, la supervisión del proceso y la validación de los resultados.

¹¹Jest – jestjs.io

¹²Playwright – playwright.dev

A lo largo del experimento, se documentan todas las decisiones tomadas, incluyendo las interacciones cuando un modelo produce resultados insatisfactorios. Este registro detallado constituye, en sí mismo, uno de los principales resultados del trabajo, ya que permite analizar no solo la calidad del software resultante sino también la eficacia del proceso de interacción con las IA.

1.4 Calidad y trazabilidad

El código fuente de los cinco proyectos está disponible públicamente en el repositorio de GitHub del trabajo [7]. Cada proyecto se organiza como un paquete independiente en el monorepo¹³, con su propia configuración de compilación, *linting* y pruebas; las referencias a fragmentos de código concretos en este documento incluyen un enlace directo al fichero correspondiente en el repositorio.

El marco de evaluación de calidad adoptado es el estándar ISO/IEC 25010 (SQuaRE) [8], complementado con las reglas de SonarQube¹⁴ —*Bugs, code smells* [9], vulnerabilidades y cobertura— como referencia para la clasificación de incidencias. Los cinco proyectos cuentan con ficheros de configuración para análisis estático con SonarQube Cloud¹⁵, lo que permite obtener métricas de complejidad ciclomática [10], deuda técnica y duplicación de código de forma automatizada.

La conformidad con el estilo de código se garantiza mediante ESLint¹⁶ configurado de acuerdo a la Guía de Estilo de Google para TypeScript¹⁷, que impone restricciones de indentación, longitud de línea, convenciones de nomenclatura y prohibición del tipo `any`. Los principios *Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation y Dependency Inversion* (SOLID) [11] se incorporan como restricciones explícitas en las plantillas de *prompts* de codificación y revisión, de modo que los modelos generan y evalúan el código atendiendo a criterios de cohesión y acoplamiento. El uso de patrones de diseño [12] es transversal a los proyectos: *Composite* y *Observer* en HANGMAN, *custom hooks* como patrón de gestión de estado en PLAYER, *Clean Architecture* en cuatro capas en CARTO y TENNIS. El análisis de calidad de cada proyecto se recoge en un informe disponible en el repositorio¹⁸ con referencias a reportes individuales de cada proyecto, y los resultados agregados se analizan en el Capítulo 7.

Los *prompts* e interacciones más relevantes de cada fase se encuentran versionados en el directorio `resources/` del repositorio [7], organizados por proyecto y fase; no se recogen la absoluta totalidad de ellos, ya que su volumen excede los límites razonables del documento y del repositorio, y las interfaces de chat empleadas no ofrecen mecanismos de exportación masiva en formato estructurado. El Capítulo 6 documenta las decisiones y problemas más relevantes de cada fase, incorporando respuestas significativas cuando resulta pertinente.

¹³Monorepo – atlassian.com/git/tutorials/monorepos

¹⁴SonarQube – sonarqube.io

¹⁵SonarQube Cloud – sonarcloud.io

¹⁶ESLint – eslint.org

¹⁷Google TypeScript Style Guide – google.github.io/styleguide/tsguide.html

¹⁸Reporte general de calidad – TFG/docs/quality_summary.md

Capítulo 2: Estado del Arte

La integración de la IA en la ingeniería del software ha evolucionado rápidamente con la aparición de los LLMs. Diversas investigaciones [13, 14] coinciden en que estos modelos están transformando la forma en que se concibe, desarrolla y mantiene el software. El presente capítulo revisa los trabajos más relevantes en torno a tres ejes temáticos: la generación de código mediante LLMs, la evaluación de la calidad del software producido por IA, y los enfoques de colaboración multiagente.

2.1 Generación de código con LLMs

La capacidad de los LLMs para generar código a partir de descripciones en lenguaje natural constituye una de las aplicaciones más estudiadas de la IA generativa en ingeniería del software. Modelos como GPT-4¹, Claude², Codex³, CodeT5⁴ o StarCoder⁵ se han integrado progresivamente en los entornos de desarrollo, ofreciendo asistencia en tiempo real para la escritura y refactorización de código.

Wang y Chen [3] presentan una revisión del panorama actual de la generación de código con LLMs, destacando su notable capacidad para producir código funcional en múltiples lenguajes. Su análisis abarca las principales arquitecturas y *benchmarks* empleados, aunque subrayan que la evaluación de la calidad del código generado no ha avanzado al mismo ritmo que las aplicaciones prácticas.

El *benchmark* más influyente en este dominio es HumanEval [15], introducido junto con Codex —el modelo subyacente a GitHub Copilot— y diseñado para medir la corrección funcional mediante la métrica *pass@k* [15]. Sus limitaciones —como problemas autocontenidos, dominio reducido o ausencia de contexto arquitectónico— son precisamente las que motivan el enfoque de este trabajo, centrado en proyectos completos en lugar de fragmentos aislados.

A pesar del impacto innegable de estas herramientas, la generación de código mediante LLMs presenta limitaciones bien documentadas. Los modelos tienden a producir código que funciona para los casos de prueba más evidentes pero que puede fallar en casos límite, presentar vulnerabilidades de seguridad o no adherirse a los principios de diseño del proyecto. Además, la calidad del resultado depende en gran medida de la calidad del *prompt*: especificaciones ambiguas o incompletas conducen invariablemente a código que requiere intervención humana significativa.

2.2 Evaluación del código generado por IA

Si bien la capacidad de los LLMs para producir código funcional ha quedado ampliamente demostrada, la calidad de ese código más allá de su corrección inmediata es una cuestión considerablemente menos resuelta.

Tosi [4] realiza una de las evaluaciones empíricas más completas hasta la fecha, comparando la calidad del código generado por GPT-3.5, GPT-4 y Google Bard en un conjunto diverso de tareas. Sus resultados confirman que, aunque estos modelos producen código funcional en la

¹GPT-4 (OpenAI) – openai.com/index/gpt-4

²Claude (Anthropic) – claude.com/product/overview

³Codex (OpenAI) – openai.com/codex

⁴CodeT5 (Salesforce) – github.com/salesforce/codet5

⁵StarCoder (BigCode) – huggingface.co/blog/starcoder

mayoría de los casos, presentan limitaciones en mantenibilidad y adherencia a buenas prácticas: el código tiende a exhibir complejidad ciclomática moderada-alta y *code smells* [9] recurrentes —métodos excesivamente largos, duplicación o acoplamiento innecesario—. Tosi concluye que la correcta definición de requisitos en el *prompt* es determinante para la calidad del resultado, y que la supervisión experta sigue siendo imprescindible.

Cabe señalar que la métrica *pass@k* [15] resulta insuficiente como indicador de calidad global: un programa puede superar todos sus casos de prueba y presentar simultáneamente alta complejidad ciclomática o deficiencia en cuanto a conformidad con principios como SOLID [11], lo que refuerza la necesidad de un análisis multidimensional como el adoptado en este trabajo.

Khade y Sambhe [13] amplían la perspectiva documentando cómo se pueden emplear herramientas de IA para la detección automatizada de defectos, el análisis estático de seguridad y la generación de conjuntos de pruebas. Esta idea —que la IA pueda generar y revisar código— es central en la metodología de este trabajo, donde se asignan modelos específicos a las tareas de revisión y *testing*, evaluando su capacidad para detectar deficiencias en código producido por un modelo distinto.

2.3 Colaboración multiagente en el desarrollo

Más allá del uso de modelos individuales para tareas aisladas, una línea emergente explora la organización de múltiples agentes de IA en flujos de trabajo colaborativos que reproduzcan la dinámica de un equipo de desarrollo humano, donde cada agente asume un rol especializado siguiendo las fases del SDLC. Turunen y Fagerholm [5] exploran la automatización en el desarrollo de aplicaciones mediante agentes como AutoGPT⁶ o Microsoft AutoGen⁷, describiendo escenarios donde distintos agentes generan código, lo revisan, producen pruebas y coordinan el flujo global. Su conclusión principal es que el futuro de la ingeniería del software podría pasar por entornos de agentes colaborativos, aunque reconocen que la coordinación efectiva entre agentes sigue siendo un problema abierto.

Dos trabajos recientes ilustran este paradigma de forma más concreta, ChatDev [14] organiza agentes especializados en un flujo de diseño, codificación y *testing* guiado por lenguaje natural, completando proyectos simples en menos de siete minutos. MetaGPT [16] va más lejos al codificar *Standard Operating Procedures* en los *prompts* de cada agente —gestor de producto, arquitecto, ingeniero, tester— exigiendo artefactos intermedios que reducen la acumulación de errores. Este enfoque presenta ventajas claras, ya que permite explotar las fortalezas complementarias de distintos modelos, introduce mecanismos implícitos de control de calidad al separar el rol de generación del de revisión, y facilita la trazabilidad del proceso mediante artefactos intermedios. Sin embargo, también plantea desafíos como la pérdida de contexto entre fases, la propagación de errores desde etapas tempranas, y las fricciones por la disparidad de los modelos.

Khade y Sambhe [13] enfatizan que la IA en el desarrollo de software no debe entenderse como un reemplazo del ingeniero humano, sino como un aumento de sus capacidades. El escenario más productivo es aquel en el que la IA automatiza las tareas más repetitivas —generación de código *boilerplate*⁸, escritura de pruebas, detección de *code smells*— mientras el ser humano aporta el juicio crítico y la visión arquitectónica. Esta aproximación es precisamente la adoptada en este trabajo.

⁶AutoGPT – agpt.co

⁷Microsoft AutoGen – microsoft.com/research/project/autogen

⁸Código *boilerplate*: Plantilla reutilizable para la estructura básica de un proyecto.

2.4 Posicionamiento del presente trabajo

La investigación existente se centra predominantemente en tareas aisladas evaluadas con *benchmarks* como HumanEval [15] o MBPP [3], dejando sin explorar el desarrollo de aplicaciones completas con arquitecturas reales. La Tabla 2.1 sintetiza el posicionamiento de este TFG en la intersección entre generación de código, evaluación de calidad y colaboración multiagente, evaluando cinco proyectos web de complejidad creciente con documentación rigurosa del proceso.

Característica	Wang [3]	Tosi [4]	Turunen [5]	TFG
Múltiples modelos de IA	No	Sí	Sí	Sí
Roles especializados por fase	No	No	Sí	Sí
Proyectos completos (no fragmentos)	No	No	No	Sí
Complejidad creciente	No	No	No	Sí
Revisión cruzada entre modelos	No	No	No	Sí
Testing generado por IA	No	No	No	Sí
Documentación de <i>prompts</i>	No	Parcial	Parcial	Amplia
Métricas de calidad de código	Teórico	Sí	No	Sí

Tabla 2.1: Comparativa entre los trabajos revisados y el presente TFG.

Capítulo 3: Modelos de IA

Este capítulo describe los modelos y herramientas de IA empleados a lo largo del trabajo, justifica su selección y documenta el cambio de enfoque metodológico introducido a partir del cuarto proyecto, como se explica en la Sección 3.3. Cada herramienta fue asignada a un rol concreto dentro del flujo de desarrollo, configurando un equipo de agentes especializados que no fue fijo desde el inicio, sino que evolucionó con la experiencia acumulada en cada proyecto hasta encontrar la combinación más adecuada para cada fase y nivel de complejidad.

3.1 Panorama actual de los LLMs en el desarrollo de software

En el momento de realización de este trabajo, el ecosistema de LLMs orientados al desarrollo de software ha alcanzado un grado de madurez suficiente para abordar tareas de complejidad real, más allá de la generación de fragmentos de código aislados. Modelos de propósito general como GPT-5 (OpenAI)¹, Claude Sonnet 4.5 (Anthropic)², Gemini 3 Pro (Google)³ o Mistral Large (Mistral AI)⁴ compiten en tareas de razonamiento sobre código, diseño arquitectónico y generación de documentación técnica. Paralelamente, herramientas en el *Integrated Development Environment* (IDE) como GitHub Copilot han evolucionado desde el autocompletado inteligente hasta un modo agente capaz de ejecutar flujos de trabajo completos de forma autónoma dentro del mismo.

Esta dualidad entre modelos conversacionales y agentes integrados resultó especialmente relevante para el presente trabajo: los primeros ofrecen flexibilidad para el diseño y la supervisión de alto nivel, mientras que los agentes operan directamente sobre el sistema de ficheros, ejecutan comandos, leen errores del compilador y realizan múltiples iteraciones de corrección sin salir del editor. Conviene señalar que el panorama de los LLMs evoluciona muy rápidamente; por ello, el análisis de este capítulo no pretende ofrecer una clasificación definitiva de modelos, sino documentar con fidelidad las herramientas concretas utilizadas, el rol asignado a cada una y el comportamiento observado en este experimento.

3.2 Modelos y herramientas utilizados

La configuración del equipo de IA no fue el resultado de una elección única al inicio del trabajo, sino de un proceso iterativo en el que la experiencia acumulada en cada proyecto determinó qué herramienta resultaba más adecuada para cada fase y nivel de complejidad. Cada modelo fue asignado al rol en que su fortaleza diferencial fuera más aprovechable y su debilidad tuviera menor impacto.

¹GPT-5 System Card – cdn.openai.com/gpt-5-system-card.pdf

²Claude Sonnet 4.5 System Card – anthropic.com/claude-sonnet-4-5-system-card

³Gemini 3 Model Card

storage.googleapis.com/deepmind-media/Model-Cards/Gemini-3-Pro-Model-Card.pdf

⁴Mistral Large Model Card

docs.mistral.ai/models/model-cards/mistral-large-2-0-24-07

3.2.1 Claude

Claude⁵ fue el designado como IA Arquitecto en los cinco proyectos. Se utilizó principalmente el modelo Claude Sonnet 4.5, disponible a través del propio chatbot *claude.ai* y como agente integrado en GitHub Copilot. Su elección para el rol arquitectónico se fundamentó en tres fortalezas: una **ventana de contexto amplia** que le permitía procesar diagramas de clases y especificaciones extensas en una única interacción; una **notable capacidad para generar documentación técnica** detallada a partir de descripciones en lenguaje natural; y una **coherencia arquitectónica** que se mantiene consistente a lo largo de conversaciones largas.

La tarea concreta de Claude en cada proyecto consistía en, partiendo de las especificaciones de requisitos y los diagramas de clases y casos de uso, generar la estructura completa del proyecto: árbol de directorios, ficheros de configuración, esqueletos de clases con métodos declarados sin implementar y documentación base. A partir del cuarto proyecto, Claude continuó en este rol integrado como agente personalizado dentro de GitHub Copilot; adicionalmente, durante el proyecto TENNIS se empleó Claude Opus 4.6 en fases de mayor exigencia de razonamiento.

3.2.2 Mistral AI

Mistral AI⁶ desempeñó el rol de IA Desarrolladora durante los tres primeros proyectos —HANGMAN, PLAYER y BALATRO—, siendo responsable de la implementación del código fuente de cada módulo a partir de los artefactos de diseño obtenidos de Claude. Su elección respondía a su buena relación entre capacidad de generar código, accesibilidad y la novedad que supuso cuando salió al mercado.

En la práctica, Mistral produjo implementaciones funcionales para módulos de complejidad media; sin embargo, a medida que los proyectos escalaron, se hicieron evidentes sus limitaciones: la gestión del estado entre módulos interdependientes resultaba inconsistente al superar ciertos límites de contexto, y la generación de patrones arquitectónicos avanzados requería numerosas iteraciones y correcciones. Estas limitaciones motivaron el cambio de enfoque descrito en la Sección 3.3.

3.2.3 Qwen AI

Qwen AI⁷ fue el agente de IA *Tester* durante los tres primeros proyectos, responsable de generar los conjuntos de pruebas unitarias con Jest. Su elección respondía a su eficacia en la generación de casos de prueba siguiendo el patrón *Arrange-Act-Assert* (AAA), con una tasa de compilación inicial elevada y cobertura de casos normales, límite y excepcionales. Al igual que Mistral, fue desplazado a partir de CARTO por la inviabilidad del flujo manual de copiar código entre interfaces chatbot para proyectos de mayor escala.

3.2.4 ChatGPT

ChatGPT⁸ fue utilizado como herramienta de apoyo en las fases iniciales de los primeros proyectos, principalmente para la redacción de especificaciones de requisitos y la exploración de alternativas arquitectónicas. Su fortaleza resultó ser su fluidez en la generación de texto técnico; sin embargo, en tareas de codificación extensa mostró tendencia a perder coherencia

⁵Claude (*chatbot*) – claude.ai

⁶Mistral (*chatbot*) – chat.mistral.ai/chat

⁷Qwen AI (*chatbot*) – qwen.ai

⁸ChatGPT (*chatbot*) – chatgpt.com

contextual en conversaciones largas, lo que limitó su utilidad. Algunos modelos GPT pudieron ser aprovechados en las fases de revisión y pruebas a través de GitHub Copilot gracias a que pudieron utilizarse de forma integrada en el IDE.

3.2.5 Google Gemini y Perplexity — Uso puntual

Google Gemini⁹ fue utilizado de forma puntual en el proyecto BALATRO para la generación de los recursos (*assets*) gráficos correspondientes a algunas de las cartas del juego, resolviendo directamente una necesidad de producción visual sin requerir diseño manual ni intervención humana. Perplexity¹⁰ fue evaluado en las fases iniciales como motor de búsqueda potenciado por IA, pero no llegó a integrarse en el flujo de trabajo al no ofrecer ventajas respecto a las herramientas adoptadas.

3.2.6 GitHub Copilot

GitHub Copilot¹¹ es una herramienta de desarrollo que funciona como una interfaz entre múltiples modelos de IA. Este asumió desde el inicio el rol de agente de IA Revisora en todos los proyectos y, en los dos últimos proyectos, —CARTO y TENNIS—, también los roles de Desarrolladora y *Tester*, consolidando en una única herramienta las responsabilidades que anteriormente se distribuían entre varias.

Su principal fortaleza frente al enfoque conversacional previo reside en la integración directa con el sistema de ficheros, gracias a su extensión de Visual Studio Code¹²: el agente lee el código existente, ejecuta comandos de compilación y estilizado (*linting*), interpreta los errores y aplica correcciones iterativas sin salir del editor. Esta capacidad, sumada a la posibilidad de definir agentes personalizados mediante ficheros de configuración en el repositorio, permitió mapear cada agente a uno de los cuatro roles —Arquitectura, Codificación, Revisión y *Testing*— con reglas y modelos específicos para cada fase. En cuanto al modelo subyacente, se empleó Claude Sonnet 4.5 para los roles de Arquitecto y Desarrolladora, y modelos de la familia GPT-5 (mini, 5.4) para los roles de Revisora y *Tester*, utilizando de forma puntual modelos más potentes como Claude Opus 4.6 y 4.7.

3.3 El cambio de enfoque metodológico

La transición de los proyectos 1–3 al proyecto 4 (CARTO) supuso un punto de inflexión, motivado por las limitaciones de escalabilidad del flujo conversacional hasta entonces empleado. En los tres primeros proyectos, la IA Desarrolladora operaba como *chatbot* web: se transfería el código al editor módulo a módulo de forma manual.

Este flujo resultaba viable para proyectos de 9 a 15 módulos, pero mostró tres limitaciones críticas al escalar: **pérdida de coherencia inter-módulo** al generar ficheros en conversaciones independientes; **fricción operativa** del ciclo manual de copiar, pegar, detectar errores y volver al *chatbot*; e **imposibilidad práctica de revisar y probar** a la escala de proyectos como CARTO, que llegó a gestionar aproximadamente 240 ficheros en cuatro capas de arquitectura. La solución fue adoptar GitHub Copilot en modo agente como herramienta principal para Codificación, Revisión y *Testing* a partir de CARTO, decisión influenciada por un curso de agentes personalizados de GitHub Copilot cursado en el ámbito de la asignatura de “*Prácticas Externas*” en simultáneo con el desarrollo del TFG.

⁹Google Gemini (*chatbot*) – gemini.google.com

¹⁰Perplexity (*chatbot*) – perplexity.ai

¹¹GitHub Copilot (*chatbot*) – github.com/copilot/agents

¹²GitHub Copilot en Visual Studio Code – code.visualstudio.com/docs/copilot/overview

Los *prompts* de los tres roles fueron rediseñados para el modelo agente. La Revisora adoptó una metodología de tres etapas: **exploración de la base de código** y generación de una lista de tareas; **revisión sistemática por grupos de ficheros** con informe de incidencias clasificadas por severidad; e **implementación de correcciones** siguiendo el informe. El *Tester* generó pruebas *end-to-end* con Playwright a partir de los diagramas de casos de uso: en una primera pasada se definían los escenarios y en una segunda se implementaban. En TENNIS, la Desarrolladora incorporó además una fase de planificación previa que categoriza los módulos y establece su orden de dependencia antes de comenzar la implementación.

3.4 Comparativa de modelos

La Tabla 3.1 sintetiza los modelos empleados: en P1–P3 el equipo se distribuía entre *chatbots* independientes; en P4–P5 los roles convergieron en GitHub Copilot con modelos de Claude y GPT como subyacentes. Ningún modelo resultó adecuado para todas las fases, lo que justifica el enfoque multiagente.

Modelo	Rol	Fortalezas	Limitaciones
Claude	Arquitecto (P1–P4)	Contexto amplio; gran coherencia; fidelidad de diseño	Sin integración directa; gran coste en modelos de mayor capacidad
Mistral	Desarrolladora (P1–P3)	Buen rendimiento en módulos de complejidad media	Pérdida de coherencia inter-módulo; dificultad con patrones avanzados
GitHub Copilot	Revisora (P1–P5); Desarrolladora y <i>Tester</i> (P4–P5); Arquitecto (P5)	Integración directa; ejecución iterativa de comandos; gestión coherente a gran escala	Uso de modelos externos con consumo de <i>tokens</i> moderado
Qwen	<i>Tester</i> (P1–P3)	Eficaz con patrón AAA; cobertura de un amplio rango de casos	Flujo manual no escalable en grandes proyectos
ChatGPT	Apoyo en requisitos (P1–P3)	Fluidez técnica; familiaridad con convenciones estándar	Pérdida de coherencia en conversaciones largas
Perplexity	Consulta puntual (ninguno)	Búsqueda en tiempo real con referencias	No aporta ventajas en desarrollo de código

Tabla 3.1: Comparativa de modelos y herramientas de IA empleados.

3.5 Uso de IA en la redacción de este documento

La redacción de esta memoria se apoyó en el modelo Claude Sonnet 4.6 como asistente de escritura LaTeX¹³, con un uso de asistencia y transformación, no generativo desde cero: el punto de partida se establece en notas propias —borradores en lenguaje llano en un cuaderno Notion¹⁴ personal— y el agente proponía reescrituras adaptadas al registro académico del TFG. Las tareas delegadas abarcaron la estructuración de capítulos, la condensación de ideas y la aplicación de convenciones de formato LaTeX —italización de términos en inglés, uso de acrónimos, ajuste de tablas—. En ningún caso operó como fuente primaria de información.

¹³[LaTeX – latex-project.org](https://www.latex-project.org)

¹⁴[Notion – notion.com/es-es](https://notion.com/es-es)

Capítulo 4: Metodología

La metodología de este TFG se sostiene sobre cuatro decisiones de diseño que se desarrollan en este capítulo: una selección de cinco proyectos web de complejidad creciente que actúan como casos de estudio (Sección 4.1); un flujo de desarrollo en cuatro fases con un modelo de IA responsable en cada una (Sección 4.2); y un conjunto de plantillas de *prompts* y una estrategia de *prompting* documentada (Sección 4.3).

4.1 Diseño experimental

Los cinco proyectos [6] se han seleccionado para que cada uno suponga un incremento de complejidad —arquitectónica, tecnológica o de escala— significativo respecto al anterior, sin saltos demasiado grandes que impidan atribuir las diferencias de comportamiento de los modelos a una variable concreta. Las Tablas 4.1 y 4.2 sintetizan, respectivamente, la caracterización funcional y las métricas de escala de cada caso de estudio; los valores de ficheros corresponden a código de producción real, excluyendo `node_modules` y artefactos de construcción.

Proyecto	Ficheros	Clases	Líneas	Conjuntos	Pruebas
HANGMAN	11	9	1 600	9	≈460
PLAYER	25	24	3 000	21	≈900
BALATRO	97	71	9 000	15	≈1400
CARTO	≈240	≈130	≈16 000	20	≈400
TENNIS	≈390	≈280	≈28 000	57	≈870

Tabla 4.1: Métricas de escala de los cinco proyectos. Ficheros, clases, líneas de código, conjuntos de pruebas y pruebas. Los valores con $\approx$ son estimaciones.

HANGMAN, inspirado en una de las prácticas de la asignatura *Programación de Aplicaciones Interactivas* del tercer curso del Grado, establece la línea base del experimento: TypeScript puro sin *framework*, arquitectura MVC con los patrones *Composite* y *Observer* [12] y once ficheros de producción que permiten calibrar el flujo sin variables de entorno adicionales. PLAYER introduce el paradigma declarativo de React y la gestión de estado asíncrono derivada de la *Application Programming Interface* (API) de audio de HTML5. BALATRO representa el mayor salto de complejidad de dominio del trío: recrea el sistema de puntuación del videojuego homónimo —manos de póker, cartas especiales (*jokers*, tarots y planetas), tienda y progresión por niveles— con persistencia en *localStorage*; al tratarse de un juego conocido, fue posible recabar retroalimentación de usuarios familiarizados con el original, lo que facilitó la detección de errores.

CARTO escala el experimento a una aplicación *fullstack*: *Clean Architecture* [2] en cuatro capas replicada tanto en el *frontend* Vue.js como en el *backend* Node.js¹, con mensajería en tiempo real sobre Socket.io² e integración con Dropbox³. Marca el punto de inflexión en el que el flujo conversacional dejó de escalar y se adoptó GitHub Copilot en modo agente como

¹[Node.js – nodejs.org/es](https://nodejs.org/es)

²[Socket.io – socket.io](https://socket.io)

³[Dropbox – dropbox.com](https://dropbox.com)

herramienta principal, decisión no planificada que surgió como respuesta a las limitaciones observadas y se documenta como hallazgo metodológico en el Capítulo 3. **TENNIS** maximiza la complejidad: gestión integral de torneos con sistema de puntuación ELO⁴, modalidad de torneos dobles con invitaciones, tres tipos de usuario con permisos diferenciados, y *frontend* Angular 19 con componentes *standalone* conviviendo en el mismo monorepo con el *backend* Express⁵. Más allá del incremento de escala, la progresión obedece a una lógica de aprendizaje: cada proyecto permitió refinar las plantillas de *prompts*, ajustar los criterios de evaluación y detectar los patrones de fallo de los modelos antes de afrontar el siguiente nivel de exigencia.

Proyecto	Función clave	Arquitectura	Pila tecnológica	Tipo de <i>testing</i>
HANGMAN	Juego del ahorcado con renderizado en <i>canvas</i>	MVC + <i>Composite</i> + <i>Observer</i>	TypeScript, Vite, Canvas API, Bulma	Jest unitario
PLAYER	Reproducción de audio, <i>playlist</i> y persistencia local	Componentes + <i>Custom Hooks</i>	React 18, TypeScript, Vite	Jest unitario
BALATRO	Juego de cartas con manos de póker, niveles y tienda	Capas (MVC + Servicios)	React 18, TypeScript, Vite	Jest unitario
CARTO	Gestión de proyectos, tareas, mensajería en tiempo real e integración Dropbox	<i>Clean Architecture</i> (4 capas)	Vue.js 3, Pinia, Socket.io, Axios, Node.js	Playwright E2E
TENNIS	Gestión integral de torneos: cuadros, <i>order of play</i> , dobles, ranking y notificaciones	<i>Clean Architecture</i> (4 capas)	Angular 19, Node.js, Express, Socket.io	Playwright E2E

Tabla 4.2: Caracterización funcional de los cinco proyectos del experimento.

4.2 Roles de IA y flujo de trabajo

El desarrollo de cada proyecto sigue un flujo en cuatro fases secuenciales inspirado en el SDLC, en el que cada fase tiene una IA responsable, artefactos de entrada y salida bien definidos y un criterio explícito de transición a la siguiente. La asignación concreta de modelos a roles y la evolución de las herramientas empleadas se abordaron en las Secciones 3.2 y 3.3 de este documento.

Fase 0 — Especificación y diseño. Antes de ejecutar las fases del flujo, se elaboran los artefactos de diseño que constituyen la entrada de la Fase 1: la especificación de requisitos del proyecto y los diagramas de clases y de casos de uso, sirviendo también de referencia durante las fases de revisión y *testing*. El contenido concreto de esta fase para cada proyecto se recoge en el Capítulo 5.

Fase 1 — Arquitectura. A partir de la especificación de requisitos y de los diagramas UML [17] de clases y de casos de uso, la IA Arquitecto produce la estructura de directorios de

⁴Puntuación ELO – ultimatetennisstatistics.com/eloRatings

⁵Express – expressjs.com

la aplicación, los esqueletos de clases sin lógica y la documentación inicial. La transición a la Fase 2 requiere que la estructura de código esté organizada y que los esqueletos incluyan firma y cabecera documentada.

Fase 2 — Codificación. La IA Desarrolladora implementa cada módulo sobre los esqueletos de la Fase 1, exigiendo conformidad con la Guía de Estilo de Google para TypeScript y los principios SOLID [11]. El criterio de transición es compilación y *linting* limpios; la operativa concreta de la Desarrolladora y su evolución entre proyectos se detalla en el Capítulo 6.

Fase 3 — Revisión de código. La IA Revisora evalúa el código según cinco criterios ponderados —Adherencia al diseño (30 %), Calidad del código (25 %), Cumplimiento de requisitos (25 %), Mantenibilidad (10 %) y Buenas prácticas (10 %)— y emite una decisión ternaria: **APPROVED**, **APPROVED WITH RESERVATIONS** o **REJECTED**. Las incidencias se clasifican por severidad y, si no se alcanza el umbral definido, sirven de entrada a una nueva iteración dedicada a arreglar las deficiencias y rehacer la revisión. La operativa concreta de la Revisora y su evolución metodológica en P4–P5 se detallan en la Sección 6.3.

Fase 4 — Testing. La IA *Tester* genera las pruebas a partir del código aprobado en la fase anterior y de los diagramas de casos de uso, siguiendo el patrón AAA —principalmente para pruebas unitarias— sobre una matriz de cobertura que exige desde casos normales hasta excepcionales. La granularidad —unitaria en P1–P3, *end-to-end* en P4–P5— se ajusta según la naturaleza del proyecto.

Rol del ingeniero supervisor. Este rol consistió en validar los artefactos de entrada y los de salida, y orquestar las transiciones entre fases, delegando en los modelos la generación de código, documentación y pruebas. La intervención directa del ingeniero se reservó a errores de integración entre sesiones, decisiones de diseño no previstas y problemas de entorno ajenos a la lógica del código, en línea con el modelo de cooperación humano-IA [13].

4.3 Plantillas y estrategia de *prompting*

Cada fase del flujo dispone de una plantilla de *prompt* reutilizable que codifica el contrato entre fases: rol del modelo, artefactos de entrada, entregables, restricciones técnicas y formato de salida. Las plantillas contienen *placeholders* que se instancian con el contexto de cada proyecto mediante un chat dedicado de Claude —*prompt-building chat*—, donde discutíamos el dominio, el modelo proponía los valores y validábamos el resultado antes de ejecutarlo. La Tabla 4.3 recoge los enlaces públicos a los cinco chats utilizados.

Proyecto	<i>Prompt-building chat</i>
HANGMAN	claude.ai/share/b55b10b1-1821-4d5e-a3b8-07852c8bc023
PLAYER	claude.ai/share/fbdecc91-fa80-4456-af18-62f7ac983df6
BALATRO	claude.ai/share/2ba985c2-4c80-48ab-a318-e0fa5c4ef493
CARTO	claude.ai/share/c244c208-d0f6-41ea-8865-c82e5251b43b
TENNIS	claude.ai/share/cb28a9f5-5399-4cb4-8213-8804c5eb1fb8

Tabla 4.3: Chats empleados para la instanciación de plantillas de *prompts*.

Las plantillas se mantienen versionadas en el directorio `resources/templates/` del repositorio del TFG [7], organizadas por fase y por versión del flujo: V1 para los proyectos conversacionales P1–P3 y V2 para los proyectos con agentes personalizados P4–P5. En estos últimos las plantillas se trasladan adicionalmente al directorio `.github/` como ficheros

.agent.md y .prompt.md. La evolución de los roles —Desarrolladora 2.0, Revisora 2.0 y *Tester* 2.0— introducida a partir de CARTO y consolidada en TENNIS se discute en la Sección 3.3 de esta memoria. la Tabla 4.4 resume las plantillas operativas y su localización en el repositorio.

Fase	Plantillas en el repositorio [7]	Patrón de <i>prompting</i>
Arquitectura V1 (P1–P3)	resources/templates/architecture/ : architecture-template.md	<i>Single-shot</i> ¹ con restricciones de estilo y cabecera obligatoria
Codificación V1 (P1–P4)	resources/templates/coding/v1/ : frontend-template.md, module-coding-template.md	<i>Single-shot</i> por módulo
Codificación V2 (P5)	resources/templates/coding/v2/ : general-codification-template.md, planning-codification-template.md	<i>Plan-then-implement</i> ² por categorías en TENNIS
Revisión V1 (P1–P3)	resources/templates/review/v1/	<i>Single-shot</i> por módulo
Revisión V2 (P4–P5)	resources/templates/review/v2/ : todo-list-review-template.md, report-review-template.md, incident-fix-review-template.md	Tres prompts encadenados: Lista de tareas por grupos → revisión → corrección
Pruebas V1 (P1–P3)	resources/templates/testing/v1/ : testing-template.md	Unitario con patrón AAA y matriz de cobertura
Pruebas V2 (P4–P5)	resources/templates/testing/v2/ : test-scenario-template.md, test-implementation-template.md	Dos prompts encadenados: escenarios e implementación

Tabla 4.4: Plantillas de *prompts* del TFG, organizadas por fase y versión.

Todas las plantillas declaran explícitamente el rol del modelo, los artefactos de entrada y el formato de salida requerido, de modo que el resultado de cada una sea directamente procesable por la siguiente, según las buenas prácticas de *prompt engineering* [18]. El encadenamiento de *prompts* dentro de una misma fase —revisión y *testing* desde CARTO, codificación desde TENNIS— fue clave para mantener la coherencia del agente a la escala de los últimos proyectos.

¹*Single-shot* Técnica de *prompting* en la que el modelo recibe toda la información necesaria en un único *prompt*.

²*Plan-then-implement* Técnica de *prompting* en la que el modelo primero genera un plan o descomposición del problema y, posteriormente ejecuta la solución siguiendo dicho plan.

Capítulo 5: Arquitectura y Diseño

Este capítulo recoge los artefactos de diseño que sirvieron de entrada al flujo de cuatro fases descrito en el Capítulo 4: la especificación de requisitos, los diagramas UML [17] de clases y de casos de uso, las decisiones arquitectónicas y las tecnologías seleccionadas para cada uno de los cinco proyectos. Puesto que la caracterización de cada proyecto y la pila tecnológica ya se han presentado en la Tabla 4.2, el énfasis de este capítulo recae sobre las decisiones de diseño y su progresión a lo largo del experimento.

5.1 Especificación de requisitos

La especificación de requisitos se abordó como un experimento en sí mismo, ensayando varios estilos de redacción y varios modelos de IA redactores para evaluar cuál ofrecía mejores resultados al alimentar a la IA Arquitecto. Se trabajó sobre los niveles de formalización habituales en los lenguajes de especificación de requisitos [19, 20, 21]: una variante informal centrada en caracterizar la aplicación libremente, una versión semiformal basada en una plantilla estructurada, y un tercer formato que varía entre proyectos: Lenguaje Natural Controlado (LNC) en sintaxis *Easy Approach to Requirements Syntax* (EARS) [22] para los tres primeros, e Historias de Usuario [23] con criterios de aceptación y prioridad para los dos últimos. Todos los documentos de especificación se encuentran versionados en el repositorio del TFG¹. La especificación semiformal —introducción, alcance, requisitos funcionales (RF) y no funcionales (RNF), resumen de actores y arquitectura técnica— se redactó con ayuda de Claude y constituye la fuente única de verdad para las fases posteriores; la variante informal se derivó de ella para mejorar la caracterización de cara a los modelos generadores de diagramas. Para el tercer formato, Mistral fue el único modelo que produjo una redacción EARS con la granularidad y coherencia necesarias, mientras que ChatGPT generó las historias de usuario de CARTO y TENNIS en formato *HU-XX* con criterios de aceptación y prioridad. El tercer formato varía según la naturaleza del proyecto: el LNC en sintaxis EARS resultó más adecuado para las aplicaciones autocontenidas de los tres primeros proyectos, mientras que las historias de usuario fueron una traducción más natural de los requisitos en lenguaje cotidiano de clientes reales para CARTO y TENNIS. En cualquier caso, la especificación semiformal fue el documento de entrada para el Arquitecto. La Tabla 5.1 resume el volumen y los aspectos diferenciadores de cada proyecto.

Proyecto	RF	RNF	Aspectos diferenciadores
HANGMAN	10	8	Arquitectura MVC, diccionario de palabras externalizable
PLAYER	20	13	Reproducción sobre API de audio HTML5, arquitectura basada en <i>custom hooks</i> , persistencia en <i>localStorage</i>
BALATRO	30	12	Evaluator de manos de póker, tres familias de cartas especiales, cinco <i>Boss Blinds</i> , economía de tienda
CARTO	30	20	Roles con permisos diferenciados, sincronización tiempo real, integración Dropbox, trazabilidad de auditoría
TENNIS	63	19	Nueve estados de inscripción ITF ¹ , doce estados de partido, clasificación ELO, notificaciones, ITF/TODS

Tabla 5.1: Volumen de la especificación de requisitos y aspectos diferenciadores.

¹[Ficheros de Especificación de Requisitos – TFG/resources/requirement-specifications/](https://github.com/TFG/requirements-specifications/)

5.2 Diagramas de clases y de casos de uso

A partir de las especificaciones, se generan para cada proyecto los diagramas de clases y los diagramas de casos de uso en formato Mermaid². La colección completa de diagramas intermedios y definitivos se encuentra en el repositorio^{3 4}, organizados por proyecto y por variante de especificación. Ambos diagramas se usan como entrada estructurada para el agente de IA Arquitecto, complementando la especificación con una representación formal del dominio y de las interacciones actor-sistema. La elección de Mermaid frente a herramientas más tradicionales se justifica por su versionado en texto plano y, sobre todo, por su capacidad de ser interpretado directamente por los modelos sin necesidad de exportar imágenes.

El diagrama de clases describe las entidades del dominio, sus atributos, sus relaciones y la cardinalidad entre ellas. Su complejidad crece de nueve clases en **HANGMAN** a más de cincuenta elementos en **TENNIS** —entidades, enumeraciones, servicios de aplicación y contratos de repositorio—, reorganizándose en capas de *Clean Architecture* a partir de **CARTO**. La Figura 5.1 muestra el diagrama UML definitivo de **HANGMAN**. Dada la complejidad de los diagramas no es viable exponerlos en este documento, pero se encuentran todos en el repositorio del proyecto y referenciados en esta memoria. El diagrama de casos de uso documenta los actores y las interacciones que soportan cada requisito. En los dos primeros proyectos basta un único actor; a partir de **CARTO** aparecen tres roles diferenciados y en **TENNIS** se elevan a cuatro al incorporar al público no registrado como actor de consulta.

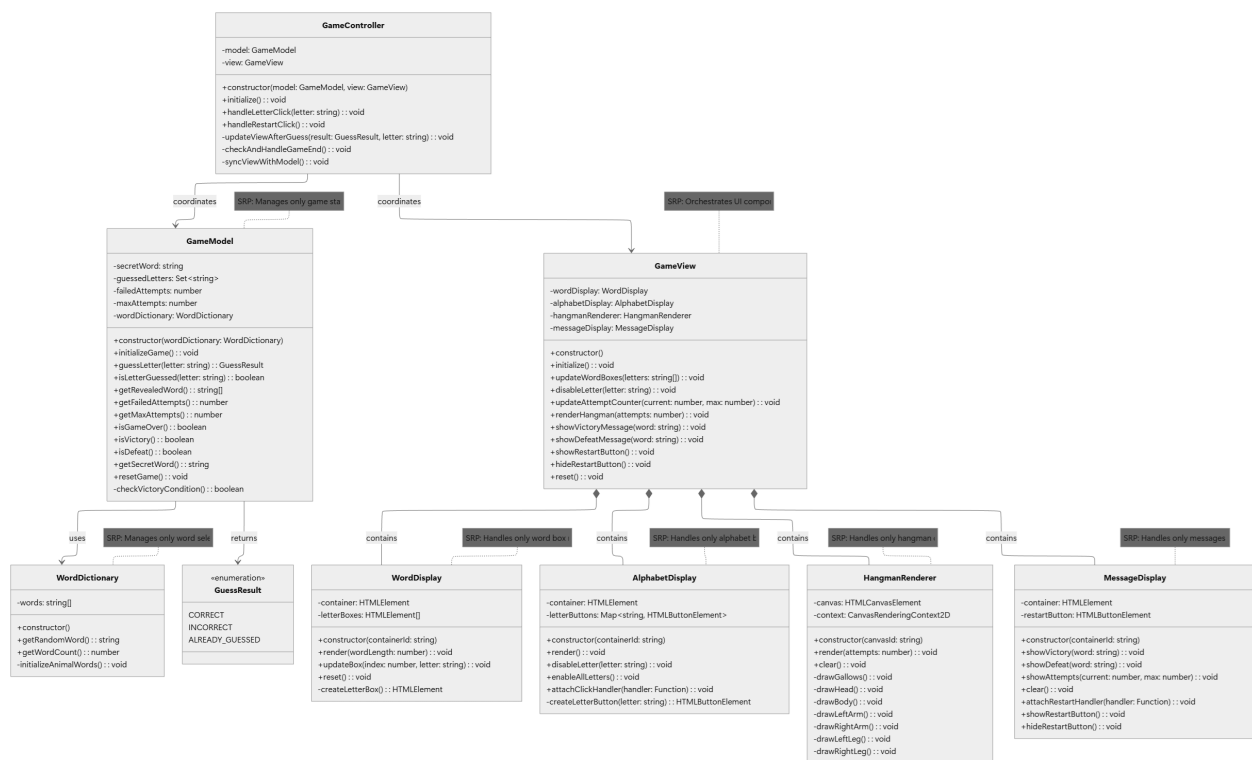


Figura 5.1: Diagrama de clases UML de **HANGMAN**, generado por Claude a partir de la especificación semiformal. ([Ver en GitHub](#))

¹Estados ITF – developer.sportradar.com/tennis/docs/ig-match-status-workflow

²Mermaid – mermaid.js.org

³Diagramas de Clases – TFG/resources/uml-class-diagrams/

⁴Diagramas de Casos de Uso – TFG/resources/uml-use-case-diagrams/

La generación de ambos tipos de diagrama fue un experimento comparativo entre modelos y variantes de especificación: para los diagramas de clases, la variante semiformal produjo resultados más ricos desde el primer intento que la informal, y el LNC no aportó ventajas adicionales sobre aquella; para los diagramas de casos de uso, Claude fue el único modelo capaz de generar sintaxis Mermaid válida y fiel a los requisitos, siendo el LNC la mejor entrada para P1–P3 y las historias de usuario para P4–P5.

5.3 Decisiones arquitectónicas

La progresión arquitectónica de los cinco proyectos aparece recogida en la Tabla 4.2 y caracterizada en la Sección 4.1; esta sección recoge las motivaciones de cada elección no derivadas del tamaño o la pila tecnológica. **HANGMAN** adopta MVC [1] con patrones *Composite* y *Observer* [12] en la vista como línea base deliberadamente conservadora. **PLAYER** pasa a componentes React con *custom hooks* porque el paradigma declarativo impone una separación presentacional-contenedor que el MVC no modela con naturalidad. **BALATRO** recupera MVC pero añade una capa de servicios transversales para evitar que la lógica de juego absorba responsabilidades ajenas. **CARTO** adopta *Clean Architecture* [2] con inversión de dependencias [11], condición necesaria para el *testing end-to-end*. **TENNIS** replica el mismo esquema *full-stack*, cada extremo con sus propias cuatro capas de *Clean Architecture*, comunicadas por la API REST y Socket.io. En todos los proyectos se aplicaron de forma transversal los principios SOLID [11] y la guía de estilo de Google.

5.4 Pila tecnológica y trazabilidad de decisiones

La Tabla 5.2 desglosa las decisiones tecnológicas complementando la Tabla 4.2. El cambio de *framework* entre proyectos es deliberado —persigue exponer a los modelos a paradigmas distintos— y cada decisión queda registrada en la especificación y reproducida en los *prompts* para evitar que los modelos introduzcan dependencias adicionales por iniciativa propia.

Proyecto	<i>Framework</i> de UI	Gestión de estado	Comunicación con backend	Estrategia de <i>testing</i>
HANGMAN	Ninguno	Estado interno	—	Jest unitario
PLAYER	React 18	<i>Custom hooks</i> + <code>localStorage</code>	—	Jest unitario
BALATRO	React 18	Servicios + <code>localStorage</code>	—	Jest unitario
CARTO	Vue 3	Pinia	Axios + Socket.io	Playwright <i>end-to-end</i>
TENNIS	Angular 19	Servicios Angular + RxJS	API REST + Socket.io	Playwright <i>end-to-end</i>

Tabla 5.2: Progresión de decisiones tecnológicas a lo largo de los cinco proyectos.

Capítulo 6: Desarrollo

Cada uno de los cinco proyectos desarrollados recorrió el flujo en cuatro fases descrito en la Sección 4.2 partiendo de los artefactos de diseño referidos en el Capítulo 5. Se documenta aquí la ejecución concreta de ese flujo organizado por fases, lo que permite observar la evolución de las herramientas y las incidencias a lo largo de la experiencia. El código fuente, los *prompts* y los informes de calidad están disponibles en el repositorio del TFG [7].

6.1 Fase de Arquitectura

En los cinco proyectos, el Arquitecto (Claude) generó en una única interacción la estructura de directorios completa y los esqueletos de clases a partir de la especificación semiformal y los diagramas UML¹. Los esqueletos reproducían con exactitud las capas arquitectónicas definidas en el Capítulo 5 y se integraron en el repositorio sin ajustes en todos los casos. La diferencia entre proyectos fue únicamente de escala: de nueve clases en HANGMAN a veinticuatro categorías de codificación en TENNIS, con las cuatro capas de *Clean Architecture* replicadas en *frontend* y *backend* para los dos proyectos *fullstack*. A partir de PLAYER, Claude incorporó de forma autónoma un *script* de inicialización de directorios, eliminando el paso de creación manual presente en HANGMAN y a partir de CARTO la integración se hacía de forma automática mediante GitHub Copilot.

6.2 Fase de Codificación

6.2.1 P1–P3: codificación modular con Mistral

En HANGMAN, Mistral codificó cada módulo de forma independiente². La incidencia más destacable surgió en la clase *WordDictionary*: en lugar de externalizar el vocabulario, el modelo lo incorporó como un *array* literal dentro del método *initializeAnimalWords()*³, introduciendo deuda de mantenibilidad que se trasladó a la fase de revisión.

En PLAYER, la codificación se articuló en dos etapas —base del *frontend*⁴ y codificación modular⁵— y se amplió con tres funcionalidades no priorizadas inicialmente: los modos *Repeat* y *Shuffle* y el control de volumen. Antes de pasar a revisión se corrigieron dos errores: la función *addSong()* del fichero *usePlaylist.ts*⁶ no actualizaba el estado correctamente, y el *useEffect()* de autoavance entre canciones en el fichero *Player.tsx*⁷ estaba roto.

En BALATRO, la complejidad del dominio determinó una articulación de ficheros distribuidos en grupos funcionales: diez *prompts* cubrieron el núcleo, las manos de póker, el sistema de puntuación, *Blinds*, las cartas especiales, los servicios, los controladores, los componentes React, el gestor de estado y las utilidades⁸. La ejecución real del juego motivó una fase de corrección previa a la revisión formal con 64 incidencias documentadas⁹. Las más representativas se

¹Estructuras de ficheros generadas – [TFG/resources/project-structures/](https://tinyurl.com/project-structures)

²*Prompts* de código HANGMAN – tinyurl.com/hangman-prompts-code

³*word-dictionary.ts*, líneas 46–54 – tinyurl.com/hangman-word-dictionary

⁴*Prompt* de *frontend* base – tinyurl.com/player-prompt-frontend

⁵*Prompts* de código PLAYER – tinyurl.com/player-prompts-code

⁶*usePlaylist.ts*, líneas 138–152 – tinyurl.com/player-use-playlist

⁷*Player.tsx*, líneas 213–243 – tinyurl.com/player-file-player

⁸*Prompts* de código BALATRO – tinyurl.com/balatro-prompts-code

⁹Registro de desarrollo de BALATRO – tinyurl.com/balatro-development-register

agrupan en cuatro áreas: en **la interfaz**, la disposición inicial era completamente vertical (véase la Figura 6.1) y se rediseñó en su totalidad, añadiendo los cuatro modales previstos en la especificación que el código no había implementado y generando los recursos gráficos de *jokers*, planetas y tarots mediante Google Gemini¹⁰; en **la lógica de juego**, se corrigió la detección de la escalera baja y se creó la clase *PermanentUpgradeJoker* para un *joker* cuyo efecto no era realizable con la jerarquía generada; en **la persistencia**, la deserialización de cartas especiales requirió el patrón *Factory* [24], ya que sin él las subclases concretas no podían reconstruirse al recargar la página; y en **los Boss Blinds**, la mecánica de *The Mouth* no había sido implementada y requirió varias iteraciones.

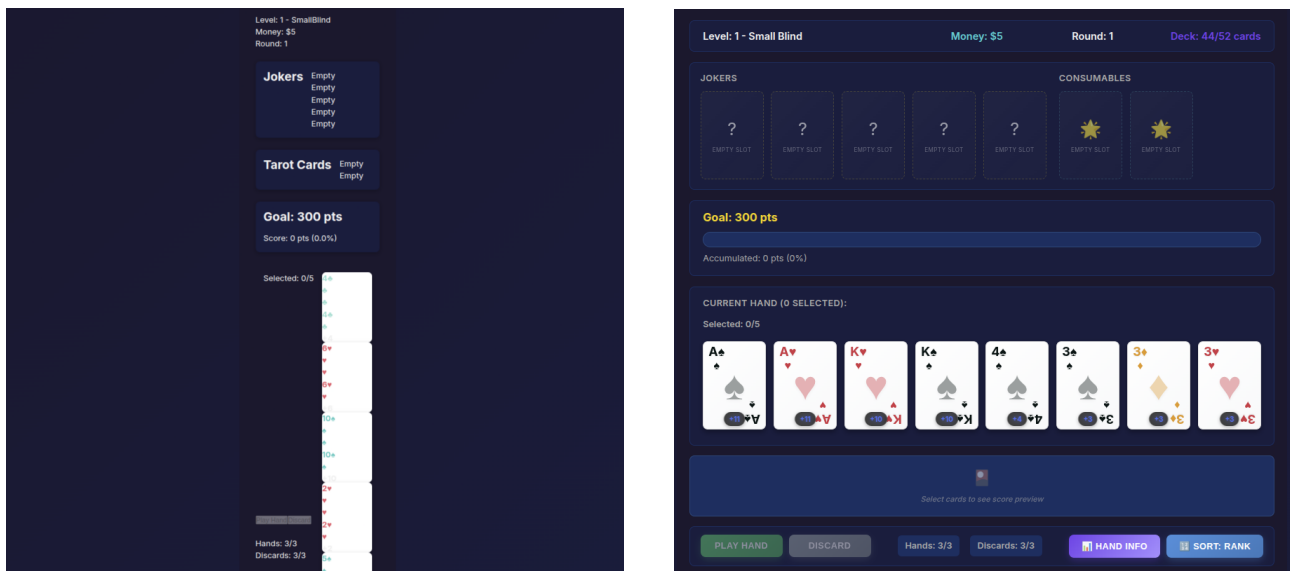


Figura 6.1: Interfaz de BALATRO antes (izquierda) y después (derecha) de la fase de corrección de errores.

6.2.2 P4–P5: codificación basada en agentes

En CARTO, el agente cubrió veintiséis grupos de módulos¹¹ y operó directamente sobre el sistema de ficheros, resolviendo de forma autónoma los errores de incoherencia intermodular. La fase de integración verificó la comunicación entre todos los módulos y la base de datos: el cliente HTTP se configuró con interceptores Axios para la inyección y renovación silenciosa de *tokens*, que se migraron al `sessionStorage` del navegador, y los siete módulos de repositorio del *frontend* se alinearon con las rutas del *backend* sin modificaciones de interfaz. El agente implementó también un sistema de notificaciones por WhatsApp mediante la API de Twilio¹² que fue descartado antes de la revisión por no encajar con el alcance del proyecto, eliminando todo el código y los campos del esquema asociados.

En el proyecto TENNIS, el agente comenzó definiendo mediante un *prompt* de planificación¹³ las 24 categorías de codificación y su orden de dependencia¹⁴ e implementó el *frontend* de forma continua¹⁵; el *backend* en Express con: 21 controladores, 14 servicios de aplicación, *middleware*

¹⁰Generación de recursos de BALATRO – gemini.google.com/share/f1833db31105

¹¹Prompts de código CARTO – tinyurl.com/cartopromptscode

¹²Twilio – twilio.com/en-us/messaging/channels/whatsapp

¹³Prompt de planificación TENNIS – tinyurl.com/tennisplanificationalprompt

¹⁴CODIFICATION-PROGRESS.md de TENNIS – tinyurl.com/tenniscodificationprogress

¹⁵Prompt de código TENNIS – tinyurl.com/tennispromptcode

de autenticación, configuración de `TypeORM`¹⁶ y servidor `WebSocket`, se generó en una segunda sesión dedicada¹⁷. Al probar la aplicación, la interfaz presentaba únicamente una página en blanco. El agente analizó la base de código y generó el fichero `IMPLEMENTATION_STATUS.md`¹⁸, que cifraba la completitud en torno al 15–20% e identificaba cinco bloqueadores: la generación de cuadros para los tres formatos de torneo, el algoritmo de cabezas de serie, la resolución de empates en clasificaciones, la gestión de los nueve estados de inscripción y el flujo de confirmación de resultados.

La corrección avanzó durante cuatro semanas. En una primera etapa se resolvieron los bloqueadores: los tres generadores de cuadros, el sistema de semillas con colocación estratégica por cuartiles, los seis criterios de desempate encadenados y el ciclo completo de confirmación de resultados con transición a estado en disputa. En una segunda etapa se implementaron las funcionalidades de mayor alcance: el módulo de Orden de Juego con asignación automática de pistas, exportación de estadísticas a cuatro formatos —PDF, Excel, CSV-ITF y TODS—, el sistema de notificaciones en tiempo real, el tablón de anuncios y los permisos de privacidad de perfiles. La funcionalidad de mayor envergadura fue el soporte para torneos de dobles: la suposición de que un *slot* de cuadro equivalía a un identificador de usuario estaba embebida en aproximadamente 30 ficheros de ambas capas, por lo que se introdujo la entidad *DoublesTeam* y se adaptaron los módulos de cuadros, clasificaciones, estadísticas y visualización. A mitad del proceso, un reescaneo sistemático de la base de código identificó requisitos menores no cubiertos que se completaron antes de cerrar la fase. El proceso evidenció dos patrones de interacción útiles: los errores de integración se resolvían más eficazmente transmitiendo hipótesis concretas al agente sobre su posible causa raíz, ya que desde el propio criterio del desarrollador humano se pudo guiar el comportamiento de la IA desarrolladora para depurar secciones concretas de la aplicación; por otro lado, las inconsistencias de estilo se corregían referenciando componentes ya implementados como modelo de referencia en lugar de tener que describir la distribución esperada en texto. La Figura 6.2 muestra el aspecto de la aplicación al principio y al final de su desarrollo.

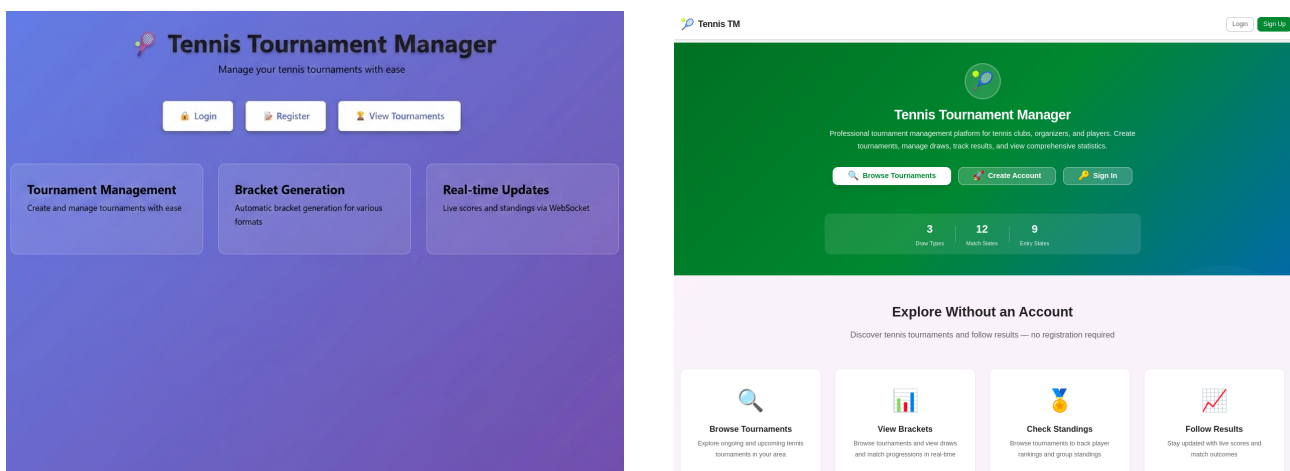


Figura 6.2: Interfaz de TENNIS antes (izquierda) y después (derecha) de la fase de corrección de estilos.

¹⁶[TypeORM – typeorm.io](https://typeorm.io)

¹⁷[Prompt de backend TENNIS – tinyurl.com/tennis-prompt-backend](https://tinyurl.com/tennis-prompt-backend)

¹⁸[IMPLEMENTATION_STATUS.md de TENNIS – tinyurl.com/tennis-implementation-status](https://tinyurl.com/tennis-implementation-status)

6.3 Fase de Revisión

6.3.1 P1–P3: revisión por módulos

En **HANGMAN**, la IA Revisora no se limitó a señalar la incidencia de *WordDictionary*: generó autónomamente el fichero `animals.json` y reescribió el constructor de la clase para cargarlo mediante importación estática, cerrando la incidencia. Fue el primer indicio de que la IA Revisora podía absorber correcciones de implementación, capacidad que el enfoque basado en agentes explotaría de forma sistemática a partir de **CARTO**. El resto de módulos recibieron veredicto aprobado en su primera revisión¹⁹.

En **PLAYER**, las indicaciones de la revisión²⁰ se aplicaron de forma manual: la IA Revisora señalaba el problema y se trasladaba a la IA Desarrolladora para corregirlo, sin generar el código de corrección de forma autónoma.

En **BALATRO**, la revisión requirió tres pasadas²¹ frente a la única pasada de los proyectos anteriores. La primera introdujo dos cambios arquitectónicos: la incorporación de identificadores únicos a las cartas *Tarot* y la externalización de los datos de balanceo a cuatro ficheros JSON (`hand-values.json`, `jokers.json`, `planets.json` y `tarots.json`), con la reescritura de la clase *BalancingConfig* para cargarlos de forma asíncrona. La segunda consolidó la lógica de la clase *HandEvaluator* —que duplicaba 160 líneas de sentencias *switch-case* entre los métodos *evaluateHand()* y *getHandType()*— y unificó el método *canActivate()* de las subclases de *Joker* en la clase base. La tercera resolvió errores de integración entre los componentes React y el resto del proyecto²².

6.3.2 P4–P5: metodología de tres etapas

En **CARTO**, la revisión²³ siguió la metodología de tres etapas descrita en la Sección 3.3: exploración y generación de lista de tareas sobre 41 grupos, revisión con producción de un informe²⁴ con 5 incidencias críticas, 30 altas, 111 medias y 79 bajas, y aplicación de correcciones prudentes. Las más relevantes fueron: **enumeraciones de dominio** que codificaban nombres de pantalla y colores propios de la capa de presentación, vulnerando la regla de dependencias de *Clean Architecture*; **secretos JWT**²⁵ sin valores de entorno obligatorios y con *fallback* a cadenas vacías; **tokens de autenticación** persistidos en `localStorage`; y **operaciones WebSocket** ejecutables sin verificar autorización.

En **TENNIS**, la misma metodología²⁶ organizó la base de código en doce grupos y produjo un informe²⁷ con 60 incidencias. Las más graves fueron: el **middleware de administración** verificaba el rol “ADMIN” mientras la enumeración real definía `SYSTEM_ADMIN` y `TOURNAMENT_ADMIN`, bloqueando *endpoints* protegidos; **seis ramas de confirmación de resultados** devolvían éxito sin persistir el cambio de estado ni notificar a los participantes; y **cinco servicios referenciados** por el contenedor de inyección de Angular sin proveedor registrado. Tras las correcciones²⁸, 58 de las 60 incidencias fueron resueltas, siendo las dos

¹⁹ *Prompts* de revisión de **HANGMAN** – tinyurl.com/hangman-prompts-review

²⁰ *Prompts* de revisión **PLAYER** – tinyurl.com/player-prompts-review

²¹ *Prompts* de revisión **BALATRO** – tinyurl.com/balatro-prompts-review

²² Resumen de revisión **BALATRO** – tinyurl.com/balatro-review-summary

²³ *Prompts* de revisión **CARTO** – tinyurl.com/carto-prompts-review

²⁴ Informe de revisión de **CARTO** – tinyurl.com/carto-review-report

²⁵ JSON Web Token – es.wikipedia.org/wiki/JSON_Web-Token

²⁶ *Prompts* de revisión **TENNIS** – tinyurl.com/tennis-prompts-review

²⁷ Informe de revisión de código de **TENNIS** – tinyurl.com/tennis-review-report

²⁸ Informe de correcciones **TENNIS** – tinyurl.com/tennis-fix-report

restantes materia de la fase de pruebas.

6.4 Fase de Pruebas

6.4.1 P1–P3: Pruebas unitarias con Jest

En **HANGMAN**, Qwen AI generó nueve conjuntos de pruebas unitarias²⁹ distribuidas en las tres capas MVC, de las cuales siete pasaron sin correcciones. Las dos excepciones compartían raíz: dependencia del entorno de Canvas bajo JSDOM. El fichero `hangman-renderer.test.ts` carecía de un *mock* del contexto 2D³⁰ y el fichero `game-view.test.ts` invocaba al método `initialize()` antes de limpiar los contadores de *mock*³¹. Ambas se resolvieron con una cobertura final del 82%.

En **PLAYER**, veintidós conjuntos de pruebas³² cubrieron componentes, *hooks*, tipos y utilidades; diecinueve pasaron correctamente. En `usePlaylist.test.ts`, la lógica de navegación con *RepeatMode.ALL* fallaba al encadenar llamadas consecutivas al método `next()` que cruzaban el límite de la lista³³; en el fichero `error-handler.test.ts`, en un principio la prueba no distinguía correctamente entre *MediaError* y errores genéricos de red³⁴. La cobertura final alcanzó el 80%.

En **BALATRO**, siete *prompts* por grupos³⁵ produjeron quince ficheros en cuatro categorías —modelos, servicios, controladores y utilidades— más el fichero de integración `game-flow.test.ts`, el primero del experimento en ejercitar la comunicación entre capas. Los conjuntos del servicio de persistencia requirieron ajustes porque los *mocks* no reproducían correctamente la deserialización ante estados incompletos; el test de integración necesitó una segunda iteración para modelar la secuencia de transiciones del ciclo de partida. Los componentes React se excluyeron del cómputo de cobertura por diseño. La cobertura alcanzó un 70%.

6.4.2 P4–P5: Playwright *end-to-end*

En **CARTO**, el agente recibió primero un *prompt* de definición de escenarios³⁶ —a partir de los diagramas de casos de uso y las rutas implementadas— y en una segunda interacción un *prompt* de implementación³⁷ que materializó esos escenarios en código con Playwright. Se generaron aproximadamente 400 pruebas en veinte conjuntos de tres niveles de criticidad; la batería crítica cubre autenticación, CRUD³⁸ de proyectos y tareas, mensajería, gestión de ficheros y notificaciones en tiempo real. Los conjuntos requirieron correcciones de configuración de entorno para que el agente pudiera sembrar y limpiar datos de prueba vía API.

En **TENNIS**, el mismo flujo de dos *prompts*^{39 40} generó 23 ficheros de pruebas en cuatro niveles de criticidad, cubriendo autenticación, gestión de torneos, cuadros, orden de juego, clasificaciones, anuncios y notificaciones. El conjunto se amplió en dos pasadas adicionales⁴¹

²⁹ *Prompts de testing* **HANGMAN** – tinyurl.com/hangman-prompts-testing

³⁰ `hangman-renderer.test.ts`, líneas 28–45 – tinyurl.com/hangman-renderer-test

³¹ `game-view.test.ts`, líneas 56–67 – tinyurl.com/hangman-game-view-test

³² *Prompts de testing* **PLAYER** – tinyurl.com/player-prompts-testing

³³ `usePlaylist.test.ts`, líneas 248–264 – tinyurl.com/player-use-playlist-test

³⁴ `error-handler.test.ts`, líneas 88–100 – tinyurl.com/player-error-handler-test

³⁵ *Prompts de testing* **BALATRO** – tinyurl.com/balatro-prompts-testing

³⁶ *Prompt* de escenarios **CARTO** – tinyurl.com/carto-prompt-scenarios

³⁷ *Prompt* de implementación **CARTO** – tinyurl.com/carto-prompt-implementation

³⁸ **CRUD** – es.wikipedia.org/wiki/CRUD

³⁹ *Prompt* de escenarios **TENNIS** – tinyurl.com/tennis-prompt-scenarios

⁴⁰ *Prompt* de implementación **TENNIS** – tinyurl.com/tennis-prompt-implementation

⁴¹ *Prompts* de ampliación de pruebas **TENNIS** – tinyurl.com/tennis-expanded-testing

derivadas de la sesión de retroalimentación (Sección 6.5).

6.5 Retroalimentación de los clientes

El cliente de CARTO valoró positivamente la aplicación —estimando una cobertura del 95% de sus expectativas— y solicitó cuatro mejoras puntuales⁴²: la **eliminación de la validación por expresión regular** del código de proyecto en favor de un formato libre; la **actualización del catálogo de tipos de proyecto** a doce categorías del dominio cartográfico, con la correspondiente migración de esquema; la **inclusión de una tarjeta en el panel de control** con indicador de tareas pendientes asignadas al usuario autenticado; y la **corrección de la integración con Dropbox**, que no reflejaba los ficheros subidos directamente ni eliminaba el directorio del proyecto al borrarlo en la aplicación. El agente implementó cada mejora en sesiones de *prompt* independientes sin modificar la arquitectura funcional existente.

El cliente de TENNIS realizó una evaluación formal de la aplicación⁴³ y señaló deficiencias en la navegación, la visualización de encuentros y la distinción entre estados de partido. El agente elaboró un plan de implementación⁴⁴ y ejecutó los cambios en sesiones sucesivas: corrigió veinticinco botones de retroceso distribuidos por toda la aplicación, estandarizó el comportamiento de navegación entre formularios e implementó el sistema de *phase-linking*⁴⁵ —vinculación de la fase clasificatoria con el cuadro principal— que automatizaba el traspaso de los jugadores de una fase a otra, ya sea del mismo torneo o entre varios.

⁴² *Prompts* de peticiones del cliente CARTO – tinyurl.com/carto-requests-client

⁴³ *Prompt* de retroalimentación del cliente TENNIS – tinyurl.com/tennis-feedback-client

⁴⁴ Plan de implementación retroalimentación TENNIS – tinyurl.com/tennis-feedback-plan

⁴⁵ *Prompt* de *phase-linking* TENNIS – tinyurl.com/tennis-prompt-phase-linking

Capítulo 7: Resultados

Los cinco proyectos desarrollados en el experimento están disponibles [6] de forma pública y accesible: los tres primeros exclusivamente siendo un *frontend* estático, y los dos últimos incluyendo *backend* desplegado en la nube. Este capítulo documenta la infraestructura de despliegue adoptada (Sección 7.1), las métricas de calidad de código obtenidas mediante SonarQube y el análisis del agente de IA (Sección 7.2), y una valoración global de los proyectos resultantes (Sección 7.3).

7.1 Despliegue de los proyectos

Los cinco proyectos se despliegan desde el mismo repositorio monorepo bajo dos estrategias según su naturaleza. Los cinco *frontends* se publican como sitios estáticos en GitHub Pages mediante un flujo de GitHub Actions que construye cada proyecto en secuencia, inyecta la variable `BASE_URL` en el fichero `vite.config.ts` de cada uno para que se sirva bajo su propio subdirectorío, y se genera una página de entrada que enlaza a los cinco subproyectos [6]. Los *backends* de **CARTO** y **TENNIS** se desplegaron en Render¹, con la base de datos alojada en Supabase². Para **CARTO** se exploró inicialmente Railway³, pero las restricciones de su plan gratuito y la incompatibilidad con la estructura monorepo motivaron el cambio. Ambos servicios se definen en el fichero `render.yaml` de la raíz del repositorio, centralizando la gestión de infraestructura en el mismo. Durante la migración de **CARTO** a Supabase, las credenciales de conexión de la base de datos fueron expuestas accidentalmente en el repositorio público a causa de un error cometido por el agente de GitHub Copilot, esa exposición fue detectada por GitGuardian⁴ y las claves fueron invalidadas y regeneradas de inmediato.

7.2 Métricas de calidad del código

La calidad del código se evaluó mediante dos herramientas complementarias. La primera es SonarQube Cloud⁵, integrado en el repositorio vía GitHub Actions con un fichero `sonar-project.properties` por proyecto. La segunda es un agente personalizado de GitHub Copilot denominado *Code Quality Agent*, generado mediante *meta-prompting* con Claude Opus 4.7, debido a su gran ventana de contexto. El agente aplica el estándar ISO/IEC 25010 [8] recorriendo las mismas métricas de calidad que SonarCloud (fiabilidad, seguridad, mantenibilidad, etc.), con el objetivo de contrastar ambas evaluaciones. Las Tablas 7.1 y 7.2 recogen las métricas obtenidas directamente de la API de SonarCloud, mientras que la Tabla 7.3 recoge el análisis complementario realizado por el agente de IA según el estándar ISO/IEC 25010. La cobertura de **CARTO** y **TENNIS** figura como no disponible porque ambos proyectos utilizan exclusivamente pruebas *end-to-end* con Playwright, cuya cobertura no es compatible con el formato LCOV de SonarCloud.

¹Render – render.com

²Supabase – supabase.com

³Railway – railway.app

⁴GitGuardian – gitguardian.com

⁵Organización del TFG en SonarQube Cloud
sonarcloud.io/organizations/tfg-fabian-gonzalez-lence

Proyecto	Bugs	Vulnerabilidades	<i>Code smells</i>	Cobertura
HANGMAN	0	0	20	95,8 %
PLAYER	0	0	81	83,3 %
BALATRO	4	2	91	72,0 %
CARTO	7	8	440	—
TENNIS	61	0	561	—

Tabla 7.1: Incidencias y cobertura de tests según SonarCloud por proyecto.

Proyecto	Fiabilidad	Seguridad	Mantenibilidad	<i>Quality Gate</i>
HANGMAN	A	A	A	OK
PLAYER	A	A	A	OK
BALATRO	B	B	A	OK
CARTO	C	C	A	OK
TENNIS	E	A	A	ERROR

Tabla 7.2: Calificaciones de calidad según SonarCloud. Fiabilidad, Seguridad y Mantenibilidad en una escala de A–E, donde A es la mejor calificación y E la peor.

Proyecto	Bloqueante	Crítica	Mayor	Menor	Total	Valoración
HANGMAN	0	0	1	5	6	Bueno
PLAYER	1	2	9	12	24	Mejorable
BALATRO	0	3	9	16	28	Mejorable
CARTO	5	6	13	6	30	Crítico
TENNIS	0	5	12	8	25	Crítico
Total	6	16	44	47	113	—

Tabla 7.3: Recuento de incidencias y valoración según el análisis del agente de IA.

Ambas herramientas convergen en la tendencia principal: la calidad decrece a medida que aumenta la complejidad del proyecto. El resultado más destacable en positivo es que la mantenibilidad obtiene calificación A en los cinco proyectos, coherente con la aplicación sistemática de los principios SOLID y la Guía de Estilo de Google para TypeScript en todas las fases del flujo. La cobertura describe una curva descendente en los tres proyectos con *testing* unitario —95,8 %, 83,3 % y 72,0 %— que refleja la complejidad creciente del dominio.

La divergencia entre las dos herramientas es especialmente reveladora en los dos proyectos *fullstack*. TENNIS acumula 61 bugs y es el único que no supera el *Quality Gate* —fiabilidad E—, mientras que el agente de IA le asigna cero incidencias bloqueantes frente a las cinco que detecta en CARTO. La explicación reside en la naturaleza de los defectos: los 61 bugs de TENNIS son errores de lógica de negocio en los estados de los partidos; las cinco incidencias bloqueantes de CARTO son problemas de diseño arquitectónico cuya gravedad requiere una comprensión global del sistema que el análisis estático no captura con la misma precisión.

La progresión de *code smells* revela que la deuda de mantenibilidad no escala linealmente: entre BALATRO y CARTO casi se quintuplican mientras los ficheros se multiplican por $\approx 2,5$, lo

que apunta a que la arquitectura *fullstack* con *Clean Architecture* genera deuda a un ritmo superior al crecimiento de la base de código. Asimismo, la revisión de **TENNIS** resolvió 58 de las 60 incidencias documentadas, pero los 61 bugs del módulo de partidos no afloraron hasta la ejecución de las pruebas *end-to-end*, evidencia de que el análisis estático no basta para garantizar la fiabilidad en flujos de negocio con estados complejos.

7.3 Análisis de los proyectos resultantes

Los cinco proyectos cumplen funcionalmente con las especificaciones de requisitos definidas en el Capítulo 5 y están accesibles en producción [6]. La degradación de calidad documentada en la Sección 7.2 es coherente con la literatura sobre generación de código con **LLMs** revisada en el Capítulo 2: la calidad decrece a medida que crece el tamaño del contexto necesario, y la progresión de escala entre proyectos reproduce ese patrón con precisión. Las incidencias de mantenibilidad —complejidad ciclomática, funciones extensas, uso recurrente de código cuestionable como el `as any`— son sistemáticas y predecibles, mientras que las de seguridad y fiabilidad —almacenamiento inseguro de credenciales, ausencia de límites de error en los árboles de componentes— solo emergen de una comprensión global del diseño del sistema, algo que los modelos actuales gestionan de forma inconsistente. Las implicaciones de este comportamiento se desarrollan en el Capítulo 8.

Capítulo 8: Conclusiones y Líneas de Trabajo Futuras

8.1 Conclusiones

Fruto del trabajo realizado, el repositorio del TFG [7] contiene el código de cinco aplicaciones web desplegadas y funcionales. El código de producción se generó en su totalidad mediante modelos de IA con roles diferenciados, capaces de producir en ocasiones, en una única iteración de *prompt*, módulos completos de código estructurado, documentado y coherente con los diagramas de diseño. Poniendo esto en perspectiva: en octubre de 2025, cuando se iniciaba el experimento con HANGMAN y se evaluaba si Claude era capaz de generar un diagrama UML en Mermaid, la posibilidad de alcanzar el resultado ahora final no era evidente. Los resultados obtenidos, analizados en el Capítulo 7, superan las expectativas iniciales en términos de escala y coherencia, aunque esta valoración positiva convive con episodios de rendimiento mediocre, particularmente en tareas iterativas —ajuste de interfaz de usuario, métricas de calidad, entre otras— donde los modelos tendían a requerir varias rondas de instrucciones para converger al resultado esperado.

Uno de los hallazgos metodológicos más relevantes del trabajo es que la distribución de roles entre modelos ha demostrado ser la decisión de diseño con mayor impacto en la calidad del resultado final. Tal como se documenta en el Capítulo 4, esta distribución introduce una capa de abstracción que reproduce la dinámica de un equipo de desarrollo humano; sin embargo, su beneficio pleno no se materializó hasta la transición —descrita en el Capítulo 3— pasando de *chatbots* conversacionales externos al IDE a agentes personalizados de GitHub Copilot integrados directamente en el repositorio. El acceso directo al código base, sin la fricción de copiar y pegar entre herramientas externas, propició un grado de especialización y desacoplamiento que redujo notablemente la incoherencia intermodular observada en los proyectos anteriores al cambio. Este cambio no fue planificado, sino que surgió de la evidencia acumulada en los tres primeros proyectos: el enfoque basado en *chatbots* conversacionales era eficaz en proyectos de pequeño alcance como HANGMAN o PLAYER, pero no escala con suficiente coherencia cuando el tamaño del sistema crece hasta la envergadura de CARTO o TENNIS, siendo BALATRO el punto de inflexión donde sus limitaciones se hicieron más evidentes.

En lo relativo al rol humano en el proceso, la experiencia confirma que el valor diferencial del ingeniero no residió en la escritura de código, sino en la coordinación entre agentes, la validación de los resultados intermedios y la traslación de las preferencias del cliente al lenguaje de *prompt* —oscilando en todo momento entre la figura de ingeniero jefe y la de portavoz del cliente—. Esta función resultó imprescindible precisamente porque los modelos presentan una limitación recurrente: cuando el contexto del proyecto supera su ventana efectiva, se producen dependencias inconsistentes, métodos duplicados o referencias a módulos inexistentes, comprometiendo la coherencia global del producto resultado. Sin la intervención humana como capa de validación y redirección, esta degradación intermodular habría afectado de forma acusada a los proyectos de mayor escala.

La elección del modelo en cada fase estuvo condicionada, en gran medida, por esta misma limitación. Claude —específicamente en sus versiones Sonnet 4.5 y 4.6, tal como se analiza en el Capítulo 3— destacó de forma consistente por combinar una gran ventana de contexto con una alta capacidad para mantener la coherencia modular y acatar las directrices técnicas establecidas en los *prompts*, convirtiéndose en el modelo más valioso a lo largo del experimento.

Otros modelos, como Mistral o Qwen, en cambio, mostraron un rendimiento notable en los proyectos de menor escala —especialmente en la generación de código y conjuntos de pruebas para **HANGMAN** y **PLAYER**—, pero se degradaron de forma significativa en **BALATRO**, donde la complejidad del modelo de dominio superó su capacidad para producir pruebas y código de manera estable. Más allá de estas diferencias entre modelos, los cinco proyectos presentan en general una estructura alineada con los diagramas de clase producidos en la fase de diseño y una documentación acorde a los estándares de ingeniería de software, lo que evidencia que el enfoque multimodelo con roles definidos, cuando se aplica correctamente, produce código de calidad más que aceptable.

La reflexión de mayor alcance que emerge de este trabajo concierne al lugar que ocupan los agentes de **IA** en el proceso de desarrollo de software. A partir de la experiencia acumulada a lo largo de los cinco proyectos, se concluye que, en el estado actual de la tecnología, estos modelos no reemplazan al ingeniero: actúan como una extensión del mismo, de manera análoga a como la aparición de los lenguajes de alto nivel no eliminó la figura del programador de bajo nivel sino que transformó su forma de trabajar [25]. La clave diferencial no reside en ser capaz de describir a una **IA** lo que se desea construir —ya que eso está al alcance de cualquier persona—, sino en poseer el conocimiento técnico suficiente para evaluar la calidad de lo que se recibe, identificar sus deficiencias y dirigir el proceso de forma que el resultado final sea correcto, mantenible y escalable. Los resultados de este TFG apuntan, en definitiva, a que el paradigma de desarrollo asistido por **IA** no elimina la necesidad de la formación técnica en ingeniería del software, sino que eleva el umbral de lo que un ingeniero bien formado puede producir en un tiempo dado.

8.2 Líneas de Trabajo Futuro

La escala del experimento, limitada a cinco proyectos web, condiciona la generalidad de las conclusiones obtenidas. Una primera línea de investigación futura sería aumentar el número de participantes y replicar el experimento con distintos perfiles de programador: pedir a varios ingenieros que construyan la misma aplicación empleando los mismos modelos y comparar los resultados permitiría aislar y estudiar el efecto de la habilidad de *prompting* del experimentador del rendimiento intrínseco del modelo.

Complementariamente, retomar el enfoque comparativo descartado en las fases iniciales del trabajo —desarrollar la misma aplicación de forma paralela con distintos modelos y comparar los resultados en términos de calidad, cobertura de requisitos y esfuerzo de supervisión— proporcionaría datos cuantitativos sobre las diferencias entre modelos que el enfoque colaborativo adoptado no permite extraer de forma aislada, y respondería preguntas que este TFG planteó pero no pudo responder empíricamente. Extender el experimento a dominios distintos del desarrollo web constituye otra dirección relevante: aplicaciones de escritorio, sistemas embebidos o *backends* de procesamiento intensivo de datos introducirían restricciones técnicas diferentes que pondrían a prueba las capacidades de los modelos en contextos menos representados en sus datos de entrenamiento y con otros lenguajes de programación.

Finalmente, la línea de mayor potencial transformador a largo plazo es la automatización completa del flujo de coordinación entre agentes: que una **IA** gestione a otras **IA**s sin intervención humana en la dirección del proceso. En el estado actual de la tecnología, el rol del jefe de desarrollo humano continúa siendo la pieza imprescindible del sistema; su sustitución por un agente orquestador autónomo representaría un avance cualitativo que, aunque técnicamente plausible, aún no ha demostrado ser viable en proyectos de complejidad real. Su investigación en el contexto del desarrollo de software asistido por **IA** constituiría una contribución de alto impacto en el ámbito de la ingeniería del software.

Capítulo 9: Conclusions and Future Lines of Work

9.1 Conclusions

As a result of the work carried out, the TFG repository [7] contains the code of five deployed and functional web applications whose production code was generated entirely by Artificial Intelligence (AI) models with differentiated roles, capable of producing, in some cases within a single prompt iteration, complete modules of structured, documented code coherent with the design diagrams. Putting this into perspective: in October 2025, when the experiment began with HANGMAN and the question was whether Claude could generate a UML diagram in Mermaid, reaching the final result was far from obvious. The results obtained, analysed in Chapter 7, exceed the initial expectations in terms of scale and coherence, although this positive assessment coexists with episodes of mediocre performance, particularly in iterative tasks—user interface adjustments, quality metrics, among others—where models tended to require several rounds of instructions to converge on the expected result.

One of the most relevant methodological findings of this work is that the distribution of roles among models proved to be the design decision with the greatest impact on the quality of the final result. As documented in Chapter 4, this distribution introduces a layer of abstraction that reproduces the dynamics of a human development team; however, its full benefit did not materialise until the transition—described in Chapter 3—from conversational *chatbots* external to the IDE to custom GitHub Copilot agents integrated directly into the repository. Direct access to the codebase, without the friction of copying and pasting between external tools, enabled a degree of specialisation and decoupling that notably reduced the inter-modular incoherence observed in the projects prior to the change. This shift was not planned; it emerged from the evidence accumulated across the first three projects: the conversational *chatbot* approach is effective for small-scale projects such as HANGMAN and PLAYER, but does not scale with sufficient coherence when the system grows to the size of CARTO or TENNIS, with BALATRO being the inflection point where its limitations became most evident.

Regarding the human role in the process, the experience confirms that the engineer’s differential value lay not in writing code, but in coordinating agents, validating intermediate results, and translating client preferences into *prompt* language—alternating at all times between the role of technical director and that of client spokesperson—. This function proved indispensable precisely because the models exhibit a recurring limitation: when the project context exceeds their effective window, inconsistent dependencies, duplicated methods, or references to non-existent modules emerge, compromising the overall coherence of the resulting product. Without human intervention as a validation and redirection layer, this inter-modular degradation would have significantly affected the larger-scale projects.

The choice of model at each phase was largely conditioned by this same limitation. Claude—specifically in its Sonnet 4.5 and 4.6 versions, as analysed in Chapter 3—consistently stood out for combining a large context window with a high capacity to maintain modular coherence and comply with the technical guidelines established in the *prompts*, making it the most valuable model throughout the experiment. Other models, such as Mistral and Qwen, on the other hand, showed notable performance in smaller-scale projects—especially in generating code and test suites for HANGMAN and PLAYER—, but degraded significantly in BALATRO, where the complexity

of the domain model exceeded its capacity to produce tests and code in a stable manner. Beyond these differences between models, the five projects generally present a structure aligned with the class diagrams produced in the design phase and documentation consistent with software engineering standards, demonstrating that the multi-model approach with defined roles, when correctly applied, yields acceptable code quality.

The broadest reflection to emerge from this work concerns the place of AI agents in the software development process. Drawing on the experience accumulated across the five projects, it is concluded that, in the current state of the technology, these models do not replace the engineer: they act as an extension of the engineer, analogously to how the emergence of high-level languages did not eliminate the low-level language programmer but transformed the way programmers work [25]. The differential key does not lie in being able to describe to an AI what one wishes to build —because that is within anyone’s reach—, but in possessing sufficient technical knowledge to evaluate the quality of what is received, identify its deficiencies, and guide the process so that the final result is correct, maintainable, and scalable. The results of this TFG ultimately suggest that the AI-assisted development paradigm does not eliminate the need for technical training in software engineering, but rather raises the ceiling of what a well-trained engineer can produce in a given time.

9.2 Future Lines of Work

The scale of the experiment, limited to five web projects, constrains the generality of the conclusions obtained. A first avenue for future research would be to increase the number of participants and replicate the experiment with different programmer profiles: asking several engineers to build the same application using the same models and comparing the results would allow isolating and studying the effect of the experimenter’s prompting skill from the intrinsic performance of the model.

Complementarily, revisiting the comparative approach discarded in the early stages of this work —developing the same application in parallel with different models and comparing results in terms of quality, requirements coverage, and supervision effort— would provide quantitative data on the differences between models that the adopted collaborative approach cannot extract in isolation, and would answer questions that this TFG raised but could not address empirically. Extending the experiment to domains beyond web development constitutes another relevant direction: desktop applications, embedded systems, or data-intensive *backends* would introduce different technical constraints that would test the models’ capabilities in contexts less represented in their training data and with other programming languages.

Finally, the line of greatest long-term transformative potential is the complete automation of the agent coordination flow: having one AI manage other AIs without human intervention in directing the process. In the current state of the technology, the role of the human development lead remains the indispensable component of the system; its substitution by an autonomous orchestrator agent would represent a qualitative advance that, although technically plausible, has not yet proven viable in projects of real complexity. Its investigation in the context of AI-assisted software development would constitute a high-impact contribution to the field of software engineering.

Capítulo 10: Presupuesto

El presupuesto de este TFG se estructura en torno a dos componentes: las horas de trabajo empleadas durante el desarrollo del proyecto y las suscripciones a los servicios de IA contratados. A diferencia de proyectos basados en infraestructura computacional, el enfoque multimodelo adoptado prescinde de hardware dedicado; el coste de capital queda reducido al de las suscripciones a los modelos utilizados.

10.1 Horas de trabajo

El coste por hora se estima en 13,50 €/h, valor representativo del salario de un desarrollador de *software* junior en España según los datos publicados en Glassdoor España¹, Talent² e Infojobs³. El proyecto se desarrolló a lo largo de nueve meses, desde la reunión inicial del 18 de septiembre de 2025 hasta la defensa prevista en junio de 2026. El plan de trabajo semanal registra 30 semanas de trabajo activo (con una semana sin avances significativos), a las que se suman aproximadamente cinco semanas adicionales de redacción de la memoria y preparación de la defensa. Considerando un ritmo medio de 14 horas semanales, propio de un proyecto desarrollado a tiempo parcial con carga intensiva en las fases de codificación, se estiman 426 horas totales. Este valor se corrobora mediante análisis automático del historial de *commits* del repositorio con el algoritmo de *git-hours*⁴: aplicando márgenes de una y dos horas por sesión de trabajo (195 sesiones identificadas), el intervalo resultante es de 368–563 horas, dentro del cual se sitúa la estimación manual. La Tabla 10.1 recoge la distribución de ese tiempo por fases del proyecto.

Tarea	Horas	Coste (€)
Investigación y revisión del estado del arte	15	203
Especificación de requisitos y diseño (5 proyectos)	80	1.080
Desarrollo asistido por IA (5 proyectos)	205	2.768
Revisión, pruebas y despliegue	66	891
Redacción de la memoria	60	810
Total	426	5.752

Tabla 10.1: Distribución estimada de horas de trabajo por fase del proyecto.

10.2 Suscripciones a servicios de IA

Los tres modelos principales se contrataron mediante planes de suscripción mensual, sin infraestructura propia ni pago por consumo de API. Los períodos activos reflejan el rol de cada herramienta en el experimento: **Claude Pro**⁵ estuvo activo los siete meses completos del trabajo, siendo el modelo central del desarrollo; **Le Chat Pro**⁶ cubrió los cuatro primeros

¹Glassdoor España – tinyurl.com/glassdoor-junior-dev-salary

²Talent.com – es.talent.com/salary?job=desarrollador+software+junior

³Infojobs – orientacion-laboral.infojobs.net/salario-desarrollador-software

⁴git-hours – github.com/kimmobrunfeldt/git-hours

⁵Anthropic Claude Pro — 18,00~€/mes – claude.com/pricing/pro

⁶Mistral AI Le Chat Pro — 14,99~€/mes (IVA no incluido) – mistral.ai/pricing

meses, coincidiendo con el uso de Mistral en P1–P3; y *GitHub Copilot Pro*⁷ se activó en marzo de 2026, al adoptar el modo agente desde CARTO. La Tabla 10.2 detalla el coste de cada suscripción.

Servicio	Proveedor	Período	Detalle	Total (€)
<i>Claude Pro</i>	Anthropic	Nov 2025 – May 2026	7 meses $\times$ 18 €/mes	126
<i>Claude Pro</i> – uso extra	Anthropic	Feb 2026	Uso prepago adicional	5
<i>Le Chat Pro</i>	Mistral AI	Nov 2025 – Feb 2026	4 meses $\times$ 15 €/mes	60
<i>Copilot Pro+</i>	GitHub	Mar 2026 – May 2026	2 facturas (\$56 $\approx$ 52 €)	52
Total				243

Tabla 10.2: Coste de las suscripciones a servicios de IA durante el proyecto.

10.3 Coste total

El coste total estimado del proyecto asciende a 5.995 €, de los cuales el componente predominante es el coste laboral. El gasto en suscripciones de IA representa un desembolso de 243 €, notablemente inferior al que implicaría el acceso a los modelos mediante API de pago por consumo. La Tabla 10.3 presenta el desglose final del presupuesto, combinando el coste laboral y el coste real de las suscripciones.

Componente	Coste (€)	Porcentaje
Horas de trabajo	5.752	95,9 %
Suscripciones a servicios de IA	243	4,1 %
Total	5.995	100 %

Tabla 10.3: Desglose total del presupuesto del proyecto.

⁷GitHub Copilot Pro+ — 39,00~\$/mes – github.com/features/copilot

Glosario

HANGMAN El Juego del Ahorcado

PLAYER Reproductor Web de Música

BALATRO Mini Balatro

CARTO Aplicación de Gestión de Proyectos Cartográficos

TENNIS Aplicación de Gestion de Torneos de Tenis

IAs Inteligencias Artificiales

IA Inteligencia Artificial

LLMs *Large Language Models*

SDLC *Software Development Life Cycle*

API *Application Programming Interface*

MVC *Model-View-Controller*

SPA *Single Page Application*

IDE *Integrated Development Environment*

SOLID *Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation y Dependency Inversion*

AAA *Arrange-Act-Assert*

TFG Trabajo de Fin de Grado

LNC Lenguaje Natural Controlado

EARS *Easy Approach to Requirements Syntax*

AI Artificial Intelligence

Bibliografía

- [1] T. Reenskaug, “A Note on the Model–View–Controller (MVC) Software Architectural Pattern,” Tech. Rep. TO-1979-34, University of Oslo, Oslo, Norway, 1979. 1, 18
- [2] R. C. Martin, *Clean Architecture: A Craftsman’s Guide to Software Structure and Design*. Prentice Hall, 2018. 1, 12, 18
- [3] J. Wang and Y. Chen, “A Review on Code Generation with Large Language Models: Application and Evaluation,” in *Proc. of the IEEE International Conference on Medical Artificial Intelligence (MedAI 2023)*, (Fuzhou, China), pp. 284–289, 2023. 2, 5, 7
- [4] D. Tosi, “Studying the Quality of Source Code Generated by Different AI Generative Engines: An Empirical Evaluation,” *Future Internet*, vol. 16, no. 6, p. 188, 2024. 2, 5, 7
- [5] J. Turunen and A. Fagerholm, “Technology Report: From Ideas to Applications,” tech. rep., University of Helsinki, Dec 2023. 2, 6, 7
- [6] F. González Lence, “TFG: Análisis de Capacidades de la IA en la Generación Semiautomática de Aplicaciones Complejas.” <https://alu0101549491.github.io/TFG-Fabian-Gonzalez-Lence/>, 2026. Página de despliegue de los proyectos del Trabajo. 3, 12, 25, 27
- [7] F. González Lence, “TFG - Análisis de Capacidades de la IA en la Generación Semiautomática de Aplicaciones Complejas,” 2026. <https://github.com/alu0101549491/TFG-Fabian-Gonzalez-Lence> – Repositorio Público del Trabajo. 4, 14, 15, 19, 28, 30
- [8] ISO/IEC, “Systems and Software Engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model,” Tech. Rep. ISO/IEC 25010:2023, International Organization for Standardization, Geneva, Switzerland, 2023. 4, 25
- [9] M. Fowler, *Refactoring: Improving the Design of Existing Code*. Boston, MA: Addison-Wesley, 1999. 4, 6
- [10] T. J. McCabe, “A Complexity Measure,” *IEEE Transactions on Software Engineering*, vol. SE-2, no. 4, pp. 308–320, 1976. 4
- [11] R. C. Martin, *Agile Software Development, Principles, Patterns, and Practices*. Upper Saddle River, NJ: Prentice Hall, 2003. 4, 6, 14, 18
- [12] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Boston, MA: Addison-Wesley, 1994. 4, 12, 18
- [13] P. S. Khade and R. U. Sambhe, “Artificial Intelligence in Software Development: A Review of Code Generation, Testing, Maintenance and Security,” *International Journal of Current Science Research and Review*, vol. 8, no. 4, pp. 1632–1641, 2025. 5, 6, 14
- [14] C. Qian *et al.*, “ChatDev: Communicative Agents for Software Development,” in *Proc. of the 62nd Annual Meeting of the ACL*, (Bangkok, Thailand), pp. 15174–15186, Association for Computational Linguistics, 2024. 5, 6

- [15] M. Chen *et al.*, “Evaluating Large Language Models Trained on Code,” 2021. 5, 6, 7
- [16] S. Hong *et al.*, “MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework,” in *Proceedings of the Twelfth International Conference on Learning Representations (ICLR 2024)*, 2024. 6
- [17] M. Fowler, *UML Distilled: A Brief Guide to the Standard Object Modeling Language*. Boston, MA: Addison-Wesley, 3 ed., 2004. 13, 16
- [18] S. Sahoo, A. Singh, S. Saha, *et al.*, “The Prompt Report: A Systematic Survey of Prompting Techniques,” 2024. 15
- [19] I. Sommerville, *Software Engineering*. Boston, MA: Pearson, 10 ed., 2015. 16
- [20] K. Pohl, *Requirements Engineering: Fundamentals, Principles, and Techniques*. Berlin, Heidelberg: Springer, 2010. 16
- [21] ISO/IEC/IEEE, “Systems and Software Engineering — Life Cycle Processes — Requirements Engineering,” Tech. Rep. ISO/IEC/IEEE 29148:2018, International Organization for Standardization, Nov 2018. 16
- [22] A. Mavin, P. Wilkinson, A. Harwood, and M. Novak, “Easy Approach to Requirements Syntax (EARS),” in *Proceedings of the 17th IEEE International Requirements Engineering Conference (RE’09)*, pp. 317–322, IEEE, 2009. 16
- [23] M. Cohn, *User Stories Applied: For Agile Software Development*. Boston, MA: Addison-Wesley, 2004. 16
- [24] M. Chochlík, “Implementing the Factory Pattern with the Help of Reflection,” *Computing and Informatics*, vol. 35, no. 1, pp. 123–139, 2016. 20
- [25] M. Welsh, “The end of programming,” *Communications of the ACM*, vol. 66, no. 1, pp. 34–35, 2023. 29, 31